

Enterprise Technical Design Document (TDD)

Project Name

Splash Financial Developer Agent Platform (SDAP)

Version: 1.0

Status: Architecture Review Draft

Author: Aditya Dubey

Architecture Style:

- Multi-Agent AI Platform
- MCP Native
- Event Driven
- Microservice Based
- Cloud Native
- Enterprise Secure

TABLE OF CONTENTS

PART I — Enterprise Overview

1. Executive Summary
 2. Business Problem
 3. Business Objectives
 4. Existing Challenges
 5. Proposed Solution
 6. Functional Requirements
 7. Non Functional Requirements
 8. Assumptions
 9. Constraints
 10. Risks
 11. Success Metrics
-

PART II — Enterprise Architecture

- 12. High Level Architecture
 - 13. Component Architecture
 - 14. Deployment Architecture
 - 15. Runtime Architecture
 - 16. Event Driven Architecture
 - 17. Service Boundaries
 - 18. AI Layer
 - 19. Security Layer
 - 20. Data Layer
-

PART III — Agent Platform

- 21. What is Developer Agent
 - 22. Why Developer Agent
 - 23. Agent Evolution
 - 24. ReAct Agent
 - 25. Deep Agent
 - 26. Loop Engineering
 - 27. Multi-Agent Architecture
 - 28. Agent Orchestration
 - 29. Memory
 - 30. Context Engineering
-

PART IV — Skills Framework

- 31. Skills
 - 32. Skill Registry
 - 33. Skill Metadata
 - 34. Skill Loading
 - 35. Skill Discovery
 - 36. Skill Versioning
 - 37. Language Skills
 - 38. Domain Skills
 - 39. Enterprise Skills
 - 40. Security Skills
-

PART V — MCP

- 41. MCP Introduction
 - 42. MCP Architecture
 - 43. MCP Client
 - 44. MCP Server
 - 45. MCP Transport
 - 46. MCP Discovery
 - 47. MCP Registration
 - 48. MCP Security
 - 49. Multi MCP Architecture
 - 50. Language Specific MCP Servers
 - 51. Enterprise MCP Servers
 - 52. Custom MCP Development
 - 53. MCP Deployment
-

PART VI — Enterprise Tools

- 54. Git
 - 55. Filesystem
 - 56. Docker
 - 57. Terminal
 - 58. Browser
 - 59. Database
 - 60. Compiler
 - 61. Testing
 - 62. Monitoring
 - 63. Logging
 - 64. Cloud
 - 65. DevOps
-

PART VII — Backend

- 66. FastAPI
- 67. PostgreSQL
- 68. Redis
- 69. JWT
- 70. OAuth
- 71. Streaming
- 72. WebSocket
- 73. Queue

- 74. Vector Database
 - 75. Observability
-

PART VIII — Frontend

- 76. React
 - 77. Vite
 - 78. Monaco
 - 79. Terminal
 - 80. Git Diff
 - 81. Agent Timeline
 - 82. Workspace
 - 83. Dashboard
-

PART IX — Security

- 84. Enterprise Security
 - 85. RBAC
 - 86. OAuth
 - 87. Secrets
 - 88. Sandboxing
 - 89. Prompt Injection
 - 90. Code Execution
 - 91. Supply Chain Security
-

PART X — Deployment

- 92. Docker
 - 93. Kubernetes
 - 94. Helm
 - 95. CI/CD
 - 96. Scaling
 - 97. Disaster Recovery
 - 98. Monitoring
 - 99. Future Roadmap
 - 100. Appendix
-

CHAPTER 1

Executive Summary

1.1 Introduction

Software development has changed significantly with the emergence of Large Language Models (LLMs). While conversational AI systems are effective for answering questions, enterprise software engineering demands autonomous systems capable of planning, reasoning, interacting with development tools, executing code, validating results, and iteratively improving solutions.

The Splash Financial Developer Agent Platform (SDAP) is proposed as an enterprise-grade AI engineering platform designed to automate substantial portions of the Software Development Life Cycle (SDLC). Unlike traditional chatbots, SDAP acts as a collaborative engineering teammate capable of understanding repositories, writing production-quality code, executing tests, reviewing changes, generating documentation, and integrating seamlessly with enterprise development ecosystems.

The platform adopts a **Multi-Agent, Model Context Protocol (MCP)-native**, and **event-driven microservices architecture**. Specialized AI agents collaborate under a central orchestration engine, leveraging standardized MCP servers to access development tools such as Git, Filesystem, Docker, Databases, Browsers, CI/CD systems, and language-specific runtimes.

By combining hierarchical planning (Deep Agents), iterative reasoning (ReAct), persistent workflow management (Loop Engineering), and secure tool execution, SDAP aims to reduce development time, improve software quality, and establish a scalable foundation for AI-assisted engineering within Splash Financial.

1.2 Vision

To build an enterprise-grade autonomous software engineering platform that functions as a trusted AI teammate capable of securely designing, implementing, testing, reviewing, documenting, and deploying production software across multiple programming languages and technology stacks.

1.3 Business Goals

- Accelerate software delivery through AI-assisted development.
 - Reduce repetitive engineering tasks.
 - Improve code quality with automated reviews and testing.
 - Standardize engineering workflows across teams.
 - Enable secure integration with enterprise development tools.
 - Support multi-language, multi-framework development.
 - Provide extensibility through MCP-based integrations.
 - Maintain enterprise security, compliance, and auditability.
-

1.4 Objectives

The platform shall:

- Understand complete code repositories.
 - Plan software implementation tasks.
 - Generate production-quality code.
 - Execute code in isolated environments.
 - Run automated tests.
 - Review and refactor generated code.
 - Produce technical documentation.
 - Interact with Git repositories.
 - Integrate with enterprise systems (Jira, Confluence, GitHub, Slack).
 - Support extensible tool integrations via MCP.
 - Operate under strict security controls.
-

1.5 Scope

In Scope

- AI-assisted software development
- Code generation
- Code review
- Automated testing
- Documentation generation
- Git operations
- Build automation
- MCP integration
- Multi-agent orchestration
- Enterprise authentication
- Tool integrations
- Language-specific execution

- Developer dashboard

Out of Scope (Phase 1)

- Autonomous production deployment without approval
- Automatic database schema migrations in production
- Direct infrastructure modification without authorization
- Unrestricted shell execution
- Self-modifying AI models

1.6 Success Criteria

The platform will be considered successful if it achieves the following measurable outcomes:

Metric	Target
Code generation accuracy	>90%
Unit test pass rate	>95%
Repository understanding time	<60 seconds
Average code review time reduction	50%
Manual development effort reduction	40%
MCP tool invocation success rate	>99%
Authentication failures	0 unauthorized accesses
Average response latency	<3 seconds for planning
Average execution latency	<30 seconds for coding tasks

1.7 Guiding Principles

1. MCP-first architecture.
2. Tool-driven, not prompt-driven.
3. Stateless services where possible.
4. Secure-by-design.
5. Human approval for critical actions.
6. Observable and auditable operations.
7. Extensible through plugins.
8. Vendor-neutral LLM integration.
9. Language-agnostic development.
10. Enterprise-grade reliability.

Chapter 2 — Business Problem Statement & Solution Vision

Document Version: 1.0

Section: Enterprise Architecture

Status: Architecture Review Draft

2. Business Problem Statement

2.1 Executive Overview

Modern software engineering organizations are increasingly constrained by growing codebase complexity, multi-language technology stacks, distributed development teams, stringent security requirements, and rapidly evolving business priorities. Although Large Language Models (LLMs) have significantly improved developer productivity through conversational assistance, they remain fundamentally limited as passive assistants rather than autonomous engineering collaborators.

Current AI coding assistants primarily generate code snippets based on prompts. They lack comprehensive repository awareness, persistent reasoning capabilities, enterprise workflow integration, structured planning, secure tool execution, and long-horizon task management. Consequently, engineers must manually coordinate planning, implementation, testing, debugging, documentation, and deployment activities.

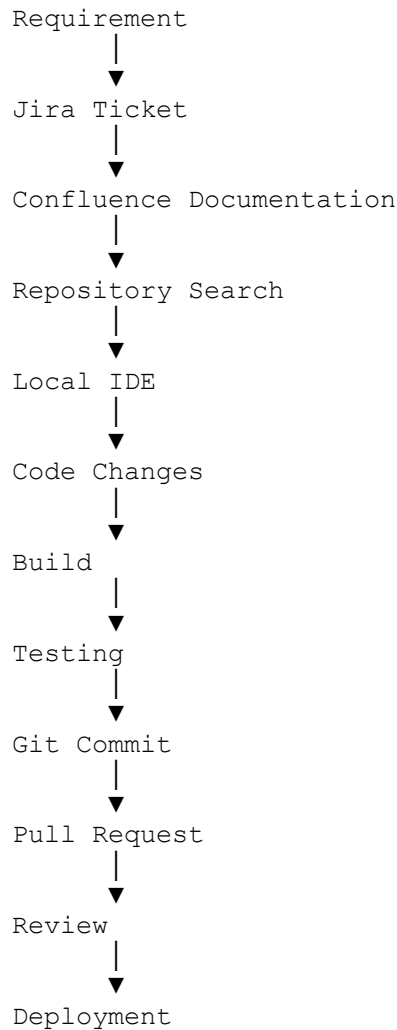
Splash Financial requires an intelligent Developer Agent Platform that moves beyond code completion to provide an enterprise-grade autonomous engineering assistant capable of understanding repositories, interacting with development tools, reasoning across multiple execution steps, collaborating with enterprise systems, and delivering production-ready software artifacts.

2.2 Current Engineering Challenges

Challenge 1 — Fragmented Development Workflow

Software engineers typically rely on a wide variety of disconnected tools throughout the software development lifecycle.

Current Workflow



Each transition introduces context switching, manual effort, and increased potential for errors.

Challenge 2 — Limited Repository Understanding

Traditional AI assistants typically process only the information included in the current prompt or a limited context window. Large enterprise repositories often contain:

- Millions of lines of code
- Hundreds of services
- Thousands of APIs
- Complex dependency graphs
- Distributed ownership
- Historical architectural decisions

Without comprehensive repository understanding, generated code often:

- violates architectural patterns,
 - duplicates existing functionality,
 - ignores dependency constraints,
 - introduces regressions,
 - or fails integration tests.
-

Challenge 3 — Lack of Planning

Existing coding assistants generally perform immediate code generation.

They rarely perform:

- requirement decomposition,
- dependency analysis,
- implementation planning,
- milestone tracking,
- risk assessment,
- execution sequencing.

Enterprise development requires structured planning before implementation.

Challenge 4 — No Persistent Memory

Conventional LLM interactions are stateless.

This results in:

- repeated explanations,
- forgotten architectural decisions,
- inconsistent implementations,
- duplicated work,
- context loss during long-running tasks.

Developer agents require persistent memory across sessions, repositories, and projects.

Challenge 5 — Manual Tool Coordination

Engineers manually invoke:

- Git
- Terminal
- Docker
- Kubernetes
- Browser
- Database
- CI/CD
- Documentation systems

Existing AI assistants cannot securely orchestrate these enterprise tools within governed execution environments.

Challenge 6 — Multi-Language Complexity

Enterprise applications commonly include:

- Python
- Java
- Kotlin
- Go
- JavaScript
- TypeScript
- React
- SQL
- Terraform
- YAML

Each ecosystem introduces unique:

- package managers,
- build systems,
- testing frameworks,
- linters,
- deployment models.

The platform must understand each language natively rather than relying on generic prompting.

Challenge 7 — Enterprise Security Requirements

Financial organizations require strict controls for:

- Authentication
- Authorization
- Audit logging
- Data governance
- Secrets management
- Code execution
- Infrastructure access

General-purpose AI assistants are not designed to satisfy these enterprise security requirements.

Challenge 8 — Long-Horizon Development

Real software engineering tasks often span:

- multiple hours,
- multiple repositories,
- dozens of files,
- hundreds of implementation steps.

Traditional conversational AI systems lose coherence over long execution horizons.

2.3 Business Drivers

The following strategic drivers motivate the proposed platform.

Accelerate Delivery

Reduce feature implementation time while maintaining engineering quality.

Improve Code Quality

Generate production-grade software adhering to organizational standards.

Reduce Operational Cost

Automate repetitive engineering activities.

Standardize Engineering Practices

Ensure every generated solution follows approved architectural patterns.

Enterprise Governance

Provide complete traceability, approval workflows, audit logs, and compliance.

Knowledge Retention

Capture engineering knowledge within reusable AI skills instead of relying solely on individual engineers.

2.4 Existing AI Landscape

Tool	Strength	Limitation
GitHub Copilot	Code completion	Limited planning
Cursor	IDE integration	Single-agent workflow
Claude Code	Excellent reasoning	Limited enterprise orchestration
OpenAI Codex	Strong coding	Limited workflow persistence
SWE-Agent	Repository automation	Research-oriented
OpenHands	Autonomous coding	Requires enterprise hardening
Devin	Long-horizon execution	Closed ecosystem

While these tools demonstrate significant advances, none fully satisfy Splash Financial's requirements for secure, extensible, MCP-native, enterprise-grade developer automation.

2.5 Proposed Solution

The proposed solution is the **Splash Developer Agent Platform (SDAP)**.

Unlike traditional coding assistants, SDAP functions as an autonomous engineering platform composed of specialized AI agents collaborating under a central orchestration engine.

High-level capabilities include:

- Repository understanding
- Intelligent planning
- Autonomous code generation
- Automated testing
- Code review
- Documentation generation
- Enterprise tool integration
- Multi-language support
- MCP-native extensibility
- Human approval workflows
- Secure execution environments

2.6 Why Multi-Agent Instead of Single Agent?

A single LLM attempting to perform planning, coding, testing, reviewing, documentation, and deployment quickly encounters context limitations and goal drift.

SDAP separates responsibilities across specialized agents.

Agent	Responsibility
Planner Agent	Requirement decomposition and execution planning
Explorer Agent	Repository understanding and dependency analysis
Coding Agent	Feature implementation
Reviewer Agent	Code review and architectural validation
Testing Agent	Automated testing and quality assurance
Documentation Agent	Technical documentation generation
Deployment Agent	CI/CD preparation and deployment workflows

This specialization improves accuracy, scalability, and maintainability.

2.7 Architectural Vision

The platform adopts the following architectural principles:

MCP-Native

All external capabilities are exposed through Model Context Protocol (MCP) servers.

Event-Driven

Agents communicate through asynchronous events rather than direct coupling.

Tool-First

LLMs reason about *which tool to use* instead of attempting unsupported operations directly.

Skill-Based

Reusable expertise is encapsulated as modular skills that can be dynamically loaded.

Secure Execution

All code execution occurs within isolated sandbox environments.

Human-in-the-Loop

High-impact operations require explicit user approval before execution.

2.8 Enterprise Value Proposition

The Splash Developer Agent Platform provides the following organizational benefits:

- Reduced software delivery timelines
 - Improved code quality
 - Standardized engineering practices
 - Reduced operational overhead
 - Secure enterprise integrations
 - Improved onboarding of new engineers
 - Reusable organizational knowledge
 - Scalable AI-assisted development
 - Vendor-neutral architecture
 - Future-proof extensibility through MCP
-

2.9 Problem-to-Solution Mapping

Current Challenge	Proposed SDAP Capability
Fragmented tooling	Unified MCP tool ecosystem
Manual repository analysis	Explorer Agent
No planning	Planner Agent
Stateless interactions	Persistent Memory & Context Engine
Manual testing	Testing Agent
Manual code review	Reviewer Agent
Documentation overhead	Documentation Agent
Multi-language complexity	Language-specific Skills + MCP Servers
Security concerns	Sandboxed execution, RBAC, OAuth2, Audit Logs
Long-running tasks	Deep Agent orchestration with persistent state

Chapter Summary

This chapter established **why** Splash Financial requires a next-generation Developer Agent Platform and identified the business and technical challenges that the architecture must address. The proposed SDAP solution introduces a secure, MCP-native, multi-agent system designed to automate the end-to-end software development lifecycle while preserving enterprise governance.

Chapter 3 — Functional & Non-Functional Requirements

3.1 Introduction

This chapter formally defines the business, technical, operational, security, and AI-specific requirements for the Splash Developer Agent Platform (SDAP).

These requirements establish the contractual baseline for architects, developers, QA teams, security engineers, DevOps engineers, and business stakeholders. Every architectural decision in subsequent chapters traces back to one or more requirements defined here.

3.2 Requirement Classification

The requirements are categorized into:

Category	Prefix	Description
Functional	FR	Business capabilities
Non-Functional	NFR	System quality attributes
Security	SR	Security & compliance
Performance	PR	Latency & throughput
Scalability	SCR	Horizontal growth
Availability	AR	Reliability & uptime
Operational	OR	Runtime operations
AI	AIR	Agent-specific capabilities
MCP	MCPR	Model Context Protocol requirements
Skills	SKR	Skills framework
Tools	TR	Tool execution
Deployment	DR	Infrastructure

3.3 Functional Requirements (FR)

FR-001 Repository Understanding

The platform shall analyze an entire software repository before implementing changes.

Acceptance Criteria

- Scan directory hierarchy
- Build dependency graph
- Detect project language(s)
- Detect frameworks
- Build symbol index
- Cache repository metadata

Priority: **Critical**

FR-002 Requirement Planning

The platform shall generate an implementation plan before modifying source code.

Plan includes:

- objectives
- impacted modules
- implementation phases
- dependencies
- risks
- estimated effort

Priority: Critical

FR-003 Autonomous Code Generation

The Coding Agent shall generate production-quality source code following organizational coding standards.

Supports:

- Python
- Java
- Kotlin
- Go

- JavaScript
- TypeScript
- React
- SQL
- Terraform
- YAML

Priority: Critical

FR-004 Repository Search

The Explorer Agent shall locate:

- classes
 - interfaces
 - methods
 - APIs
 - configurations
 - dependencies
 - environment variables
-

FR-005 Dependency Analysis

The platform shall generate dependency graphs identifying:

- internal libraries
 - external packages
 - circular dependencies
 - unused dependencies
 - transitive dependencies
-

FR-006 Refactoring

The platform shall safely refactor:

- classes
- packages
- modules
- methods

- variable names

while preserving application behavior.

FR-007 Test Generation

Automatically generate:

- Unit Tests
 - Integration Tests
 - API Tests
 - UI Tests
 - Regression Tests
-

FR-008 Documentation Generation

Automatically generate:

- README
 - API documentation
 - Sequence diagrams
 - Architecture diagrams
 - Release notes
 - Changelogs
-

FR-009 Git Integration

Support:

- clone
 - checkout
 - branch
 - commit
 - merge
 - rebase
 - push
 - pull request generation
-

FR-010 Human Approval

Before executing high-risk operations, obtain user approval.

Examples:

- deleting files
 - production deployment
 - database migration
 - infrastructure modification
-

FR-011 Conversation Memory

Maintain persistent memory across sessions.

Memory includes:

- architectural decisions
 - coding preferences
 - repository context
 - previous implementations
-

FR-012 Multi-Agent Collaboration

Support collaboration between:

Planner

↓

Explorer

↓

Coder

↓

Reviewer

↓

Tester



Documentation



Deployment

FR-013 Multi-Model Support

Support:

- OpenAI
 - Anthropic
 - Groq
 - Gemini
 - Azure OpenAI
 - Local LLMs
-

FR-014 Dynamic Model Switching

Switch providers during execution based on:

- latency
 - cost
 - reasoning complexity
 - token limits
-

FR-015 Streaming Responses

Support:

- token streaming
- tool progress
- execution logs
- terminal output

FR-016 Multi-Workspace Support

Each workspace maintains isolated:

- repositories
- conversations
- settings
- skills
- memory

FR-017 Plugin Architecture

Support dynamic plugin installation.

FR-018 Audit Logging

Log every:

- prompt
- tool call
- MCP request
- model response
- approval
- deployment

FR-019 Code Review

Reviewer Agent checks:

- architecture
- naming
- complexity
- security
- style
- maintainability

FR-020 Deployment Assistance

Generate:

- Dockerfile
- Helm Charts
- Kubernetes YAML
- Terraform
- CI/CD Pipelines

3.4 AI Requirements (AIR)

AIR-001

Reason before acting.

AIR-002

Never hallucinate repository files.

AIR-003

Use tools instead of assumptions.

AIR-004

Verify execution results.

AIR-005

Retry failed operations intelligently.

AIR-006

Maintain execution state.

AIR-007

Support Deep Agent orchestration.

AIR-008

Support ReAct reasoning.

AIR-009

Support Loop Engineering.

AIR-010

Self-reflect before final response.

3.5 MCP Requirements (MCPR)

MCPR-001

The platform shall support MCP Version 1.x.

MCPR-002

Support STDIO transport.

MCPR-003

Support Streamable HTTP.

MCPR-004

Support SSE.

MCPR-005

Support OAuth.

MCPR-006

Support Tool Discovery.

MCPR-007

Support Resource Discovery.

MCPR-008

Support Prompt Discovery.

MCPR-009

Support multiple simultaneous MCP servers.

MCPR-010

Support dynamic server registration.

MCPR-011

Support custom enterprise MCP servers.

MCPR-012

Support parallel tool execution.

3.6 Skills Requirements (SKR)

SKR-001

Every language shall have its own Skill Package.

Example

Python Skill

Java Skill

React Skill

Go Skill

Rust Skill

SKR-002

Skills shall contain:

- metadata
 - prompts
 - templates
 - workflows
 - examples
 - tool mappings
-

SKR-003

Skills shall be dynamically loadable.

SKR-004

Support skill inheritance.

SKR-005

Support skill versioning.

SKR-006

Support enterprise custom skills.

3.7 Tool Requirements (TR)

The platform shall integrate:

✓ Filesystem

✓ Git

✓ GitHub

✓ GitLab

✓ Docker

✓ Kubernetes

✓ Browser

✓ PostgreSQL

✓ MongoDB

✓ Redis

✓ Python

✓ Java

✓ Node

✓ Go

✓ Rust

✓ Terminal

✓ Slack

✓ Jira

✓ Confluence

✓ AWS

✓ Azure

✓ GCP

✓ Playwright

✓ Cypress

✓ Terraform

3.8 Security Requirements (SR)

Authentication

OAuth2

JWT

SAML

SSO

Authorization

RBAC

ABAC

Workspace Isolation

Secrets

Vault

AWS Secrets Manager

Azure Key Vault

Execution

Sandbox

Docker Isolation

Filesystem Isolation

Read-only mode

Compliance

SOC2

PCI DSS

GDPR

ISO27001

Audit

Every operation must be logged.

3.9 Performance Requirements (PR)

Requirement	Target
Initial Planning	<5 sec
Repository Scan	<60 sec
Tool Call	<2 sec
Model Routing	<500 ms
MCP Discovery	<3 sec
Code Generation	<30 sec
Unit Test Generation	<45 sec

Requirement	Target
Streaming Latency	<200 ms

3.10 Scalability Requirements (SCR)

Support:

10 concurrent users (development)

↓

100 users (pilot)

↓

1,000 users (production)

↓

10,000+ users (enterprise)

Support:

Horizontal scaling

Stateless APIs

Distributed caches

Message queues

Multi-region deployment

3.11 Availability Requirements (AR)

Metric	Target
Availability	99.9%
MTTR	<30 min
Backup Frequency	Hourly
Disaster Recovery	<2 hours

Metric	Target
Data Loss	<5 min

3.12 Operational Requirements (OR)

The platform shall support:

- Structured logging
 - OpenTelemetry tracing
 - Prometheus metrics
 - Grafana dashboards
 - Health checks
 - Readiness probes
 - Liveness probes
 - Cost monitoring
 - Token monitoring
 - Budget alerts
-

3.13 Traceability Matrix (Excerpt)

Requirement Architecture Component

FR-001	Explorer Agent
FR-002	Planner Agent
FR-003	Coding Agent
FR-007	Testing Agent
FR-009	Git MCP Server
FR-011	Memory Service
AIR-007	Deep Agent Orchestrator
MCPR-009	MCP Client Manager
SKR-003	Skill Loader
TR-001	Tool Registry
SR-001	Auth Service
PR-001	LLM Router
OR-001	Observability Stack

Chapter Summary

This chapter establishes the **contractual requirements** that govern the entire SDAP architecture. These requirements will directly influence the design of the agent orchestration engine, skills framework, MCP integration layer, tool ecosystem, security model, and deployment architecture.

Chapter 4 — Enterprise Architecture Overview

4.1 Introduction

This chapter establishes the enterprise architecture for the Splash Developer Agent Platform (SDAP). It defines the system's logical decomposition, architectural principles, runtime interactions, service boundaries, communication mechanisms, and technology stack.

Unlike traditional AI applications, SDAP is **not** a chatbot. It is a distributed AI engineering platform composed of autonomous agents, orchestration services, Model Context Protocol (MCP) integrations, enterprise tools, persistent memory, and governance components.

The architecture is designed around six core objectives:

- Extensibility
 - Security
 - Observability
 - Scalability
 - Maintainability
 - AI-native orchestration
-

4.2 Architecture Principles

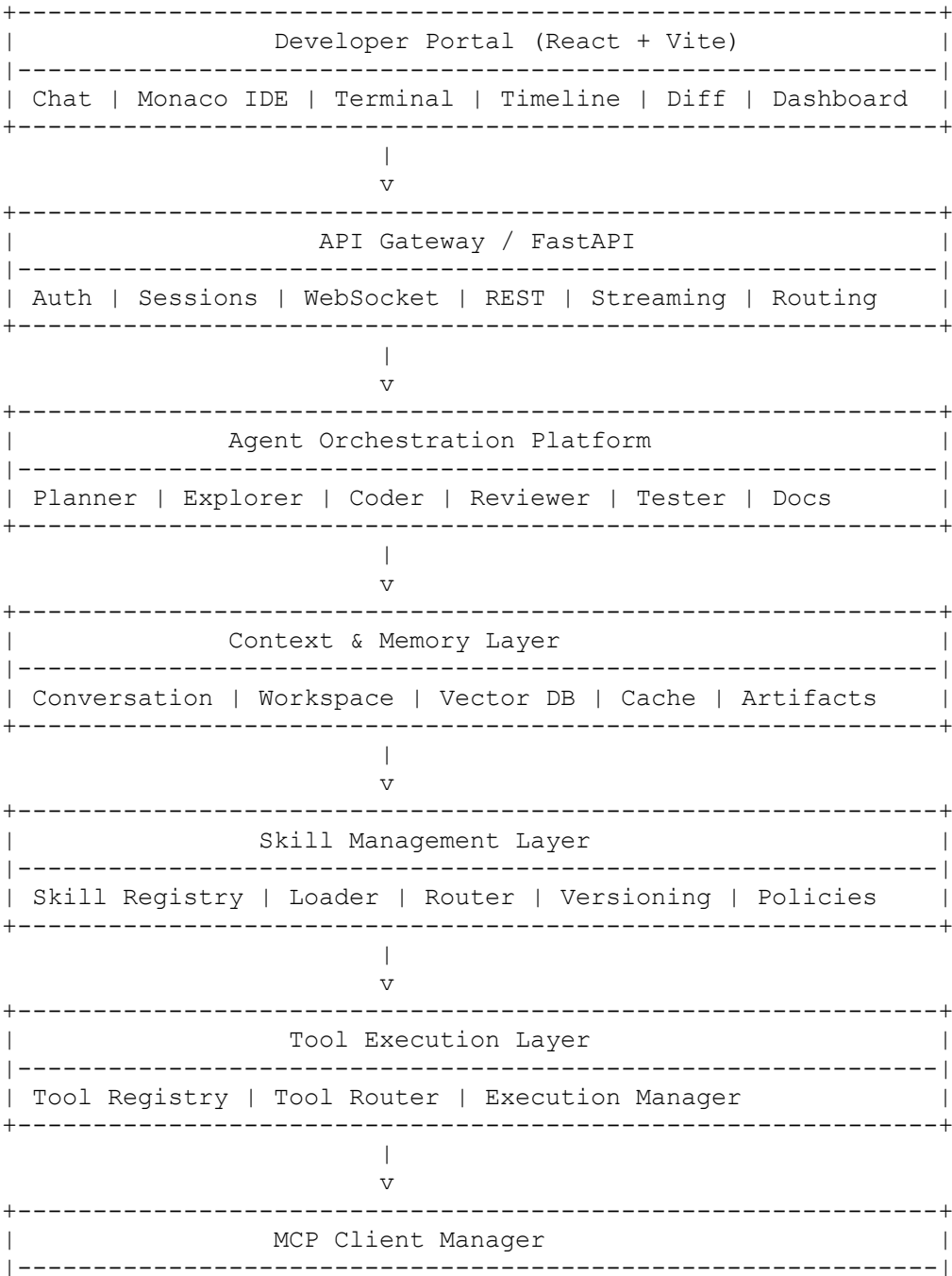
The following principles guide every design decision.

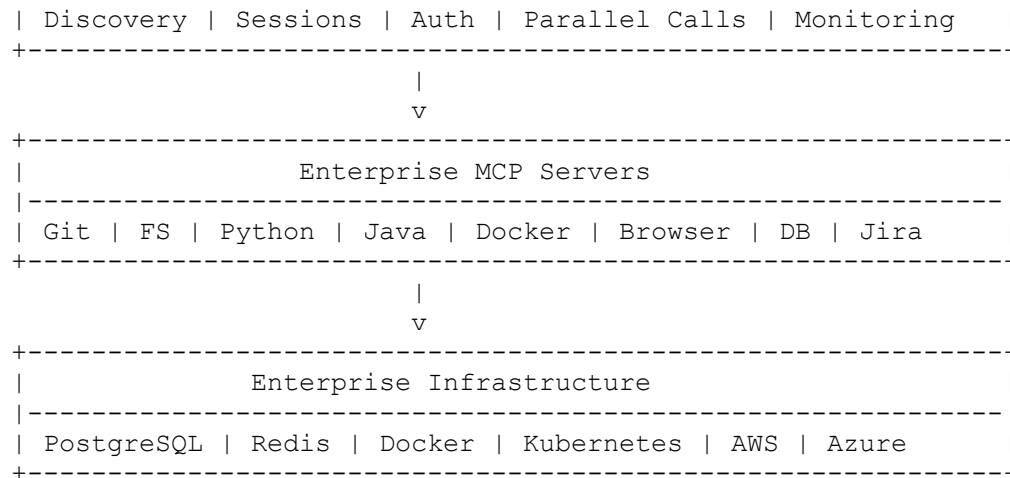
Principle	Description
AI-First	AI agents drive workflows, not hardcoded business logic
MCP-Native	All external capabilities are exposed through MCP servers
Tool-First	Agents execute tools instead of relying on hallucinated knowledge
Modular	Every service is independently deployable
Event-Driven	Components communicate asynchronously where appropriate
Stateless	APIs remain stateless; state is externalized
Secure-by-Design	Authentication, authorization, sandboxing, and auditing are built in
Observable	Every interaction is logged, traced, and measurable
Human-in-the-Loop	Sensitive operations require explicit approval

Principle	Description
Vendor Neutral	Multiple LLM providers are supported

4.3 High-Level System Architecture

The SDAP architecture is divided into **nine logical layers**.





4.4 Layer Responsibilities

Layer 1 — Presentation Layer

Responsibilities

- Chat interface
- Monaco code editor
- Repository explorer
- Git diff viewer
- Terminal
- Agent execution timeline
- Logs
- Dashboard
- Notifications

Technology

- React
- Vite
- TypeScript
- Redux Toolkit
- Monaco Editor
- Tailwind CSS
- WebSockets

Layer 2 — API Gateway

The API Gateway acts as the single entry point.

Responsibilities:

- Authentication
- Authorization
- Session Management
- Rate Limiting
- Streaming Responses
- WebSocket Management
- Request Validation
- API Versioning

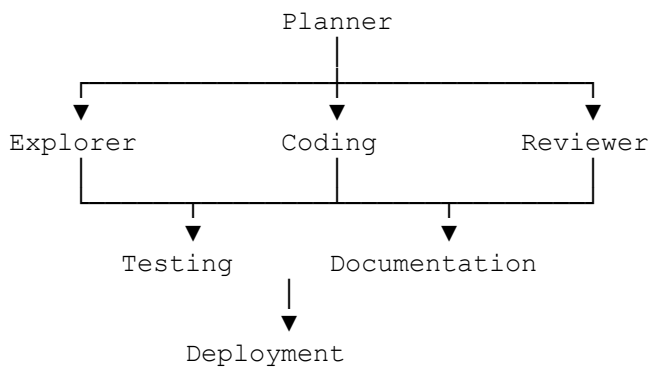
Technology:

- FastAPI
 - Uvicorn
 - JWT
 - OAuth2
-

Layer 3 — Agent Runtime

The heart of SDAP.

Contains specialized autonomous agents.



Each agent:

- has isolated prompts,
- its own tools,
- dedicated skills,
- independent memory,
- execution policies.

4.5 Why Multi-Agent?

Instead of one large AI handling everything, responsibilities are separated.

Benefits:

- Smaller prompts
 - Better reasoning
 - Easier testing
 - Independent upgrades
 - Parallel execution
 - Reduced hallucinations
 - Specialized optimization
-

4.6 Agent Responsibilities

Agent	Responsibility
Planner	Requirement decomposition
Explorer	Repository analysis
Coding	Code implementation
Reviewer	Code quality
Testing	Test execution
Documentation	Documentation generation
Deployment	Build & release preparation

4.7 Context & Memory Layer

Memory is treated as a first-class architecture component.

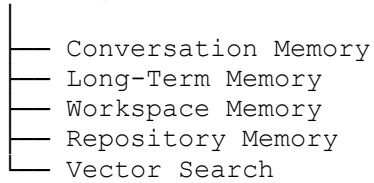
Stores:

- Conversations
- Repository embeddings
- Design decisions
- User preferences
- Previous executions

- Tool outputs
- Generated artifacts

Components

Context Manager



Technology:

- PostgreSQL
- pgvector
- Redis
- Object Storage

4.8 Skills Layer

Skills encapsulate reusable expertise.

A skill contains:

- Instructions
- Best practices
- Templates
- Validation rules
- Tool mappings
- Example workflows

Examples:

- Python Skill
- React Skill
- Spring Boot Skill
- Security Skill
- SQL Skill

Detailed Skills Framework is covered in Chapter 7.

4.9 Tool Execution Layer

The Tool Layer abstracts execution from reasoning.

Instead of directly invoking tools, agents submit requests to a centralized Tool Execution Manager.

Responsibilities:

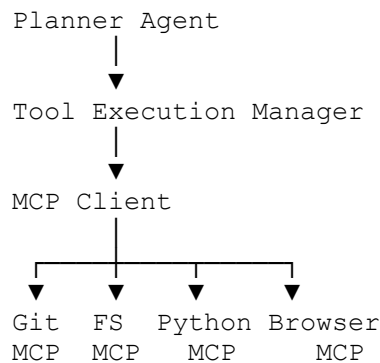
- Tool discovery
 - Authorization
 - Validation
 - Retry logic
 - Timeout management
 - Result normalization
 - Audit logging
-

4.10 MCP Client Manager

The MCP Client Manager is the bridge between SDAP and external capabilities.

Responsibilities:

- Connect to MCP servers
- Discover available tools
- Authenticate
- Maintain sessions
- Execute tools
- Handle failures
- Support parallel execution
- Cache capabilities



4.11 Enterprise MCP Servers

SDAP will support both **built-in** and **custom** MCP servers.

Language Servers

- Python
- Java
- Go
- Rust
- Node.js
- TypeScript
- React
- .NET

Development Servers

- Git
- GitHub
- GitLab
- Filesystem
- Terminal
- Docker
- Kubernetes

Database Servers

- PostgreSQL
- MySQL
- MongoDB
- Redis

Browser Automation

- Playwright
- Chromium

Enterprise Servers

- Jira
- Confluence
- Slack
- Microsoft Teams

Each server will be fully documented in the MCP chapter (Chapter 8).

4.12 Data Layer

Persistent storage components:

Storage	Purpose
PostgreSQL	Metadata, users, workspaces
Redis	Cache, sessions, queues
pgvector	Semantic search
Object Storage	Logs, artifacts, files

4.13 Cross-Cutting Concerns

Every layer integrates with these platform services:

Authentication

- OAuth2
- JWT
- SSO

Authorization

- RBAC
- Workspace isolation
- Policy engine

Observability

- OpenTelemetry
- Prometheus
- Grafana
- Loki

Security

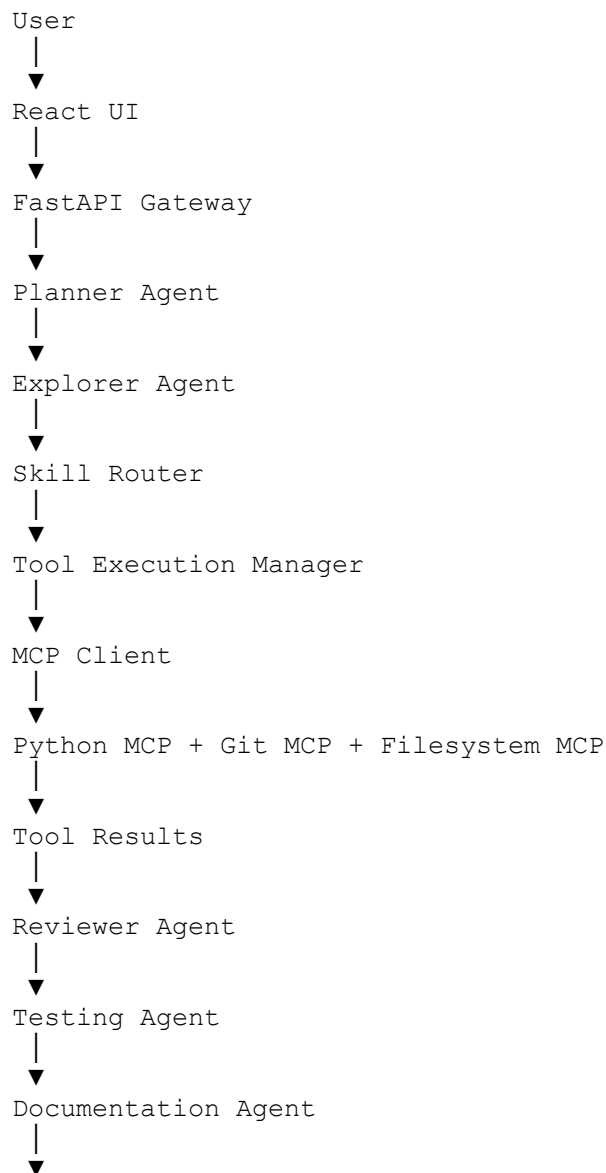
- Secrets Manager
- Audit Logs
- Encryption

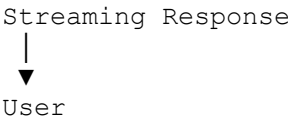
- Sandboxing

Governance

- Cost tracking
 - Token tracking
 - Approval workflows
 - Policy enforcement
-

4.14 End-to-End Request Flow





4.15 Technology Stack

Layer	Technology	Justification
Frontend	React + Vite	Fast, modular UI
Backend	FastAPI	Async, high-performance APIs
Agent Runtime	LangGraph (recommended)	Stateful multi-agent orchestration
LLM SDK	OpenAI Agents SDK (evaluation) / LiteLLM	Multi-model abstraction
MCP	Official MCP SDK + FastMCP	Standards-compliant tool integration
Database	PostgreSQL	Relational metadata
Vector Store	pgvector	Unified semantic search
Cache	Redis	Low-latency state
Container	Docker	Sandboxed execution
Orchestration	Kubernetes	Horizontal scaling
Messaging	Redis Streams / RabbitMQ	Async workflows
Observability	OpenTelemetry + Prometheus + Grafana	Production monitoring

4.16 Architectural Decisions (ADR Summary)

ADR	Decision	Rationale
ADR-001	Multi-agent architecture	Separation of responsibilities
ADR-002	MCP-native integrations	Standardized tool interoperability
ADR-003	Tool-first execution	Reduce hallucinations and improve reliability
ADR-004	Event-driven workflows	Better scalability and resilience
ADR-005	FastAPI backend	Async performance and Python ecosystem
ADR-006	React + Vite frontend	Modern developer experience
ADR-007	PostgreSQL + pgvector	Unified transactional and semantic storage
ADR-008	Docker sandbox	Secure code execution

Chapter Summary

This chapter establishes the **high-level architecture** of SDAP, defining the layered design, major components, technology choices, and interaction model. It serves as the blueprint for all detailed subsystem designs that follow.

Agent Runtime & Orchestration Architecture

5.1 Introduction

The Agent Runtime is the **brain** of the entire SDAP platform.

Unlike conventional AI assistants that execute a single prompt-response cycle, SDAP implements a persistent, stateful, event-driven orchestration engine responsible for planning, coordinating, executing, monitoring, and recovering long-running engineering workflows.

The runtime manages the complete lifecycle of AI agents, enabling collaboration among specialized agents while maintaining consistency, security, observability, and fault tolerance.

5.2 Why an Orchestrator?

Without orchestration, AI systems typically suffer from:

- Unstructured reasoning
- Duplicate work
- Tool misuse
- Hallucinations
- Infinite loops
- Context explosion
- Poor recovery
- No workflow visibility

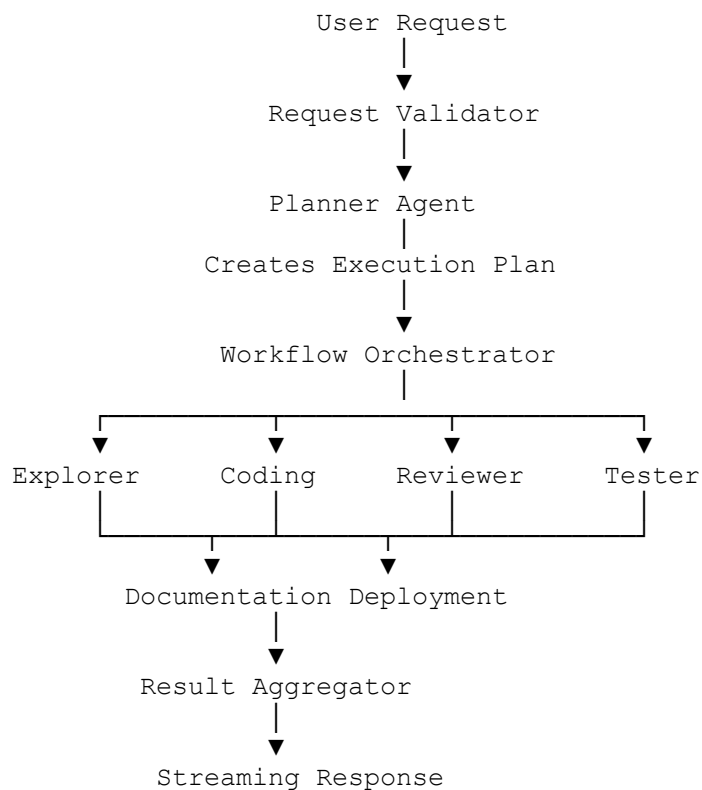
The Agent Orchestrator addresses these issues by introducing structured execution.

Responsibilities include:

- Task planning
- Agent scheduling
- Dependency resolution
- Context propagation
- Tool routing
- Skill loading
- Memory synchronization

- Approval management
 - Retry handling
 - Failure recovery
 - Progress tracking
 - Cost optimization
-

5.3 Runtime Architecture



5.4 Runtime Components

1. Request Validator

Responsibilities

- Validate authentication
- Validate permissions
- Validate workspace
- Validate repository
- Validate prompt size

- Validate budget
- Validate policies

Input

```
{  
  "workspace": "loan-service",  
  "repository": "credit-api",  
  "task": "Add OAuth support"  
}
```

Output

Validated execution request

2. Planner Agent

The planner converts natural language into an executable workflow.

Input

Implement OAuth Login

Output

1. Explore repository
2. Locate auth module
3. Identify JWT implementation
4. Create OAuth layer
5. Update APIs
6. Generate tests
7. Review
8. Generate documentation
9. Create Pull Request

Planner never writes code.

Its responsibility is planning only.

3. Workflow Engine

The Workflow Engine is responsible for:

Task Queue

Dependencies

Scheduling

Retries

Parallel Execution

Checkpointing

Human Approval

Timeouts

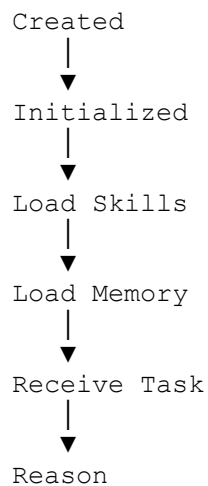
Cancellation

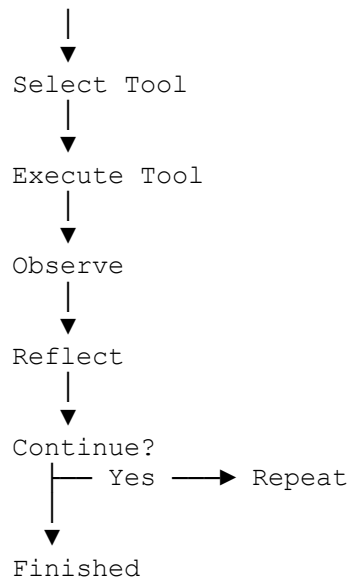
Recovery

It is essentially the operating system of the Developer Agent.

5.5 Agent Lifecycle

Every agent follows the same lifecycle.





This lifecycle is identical for:

Planner

Explorer

Coder

Reviewer

Tester

Documentation

Deployment

5.6 Execution State Machine

NEW

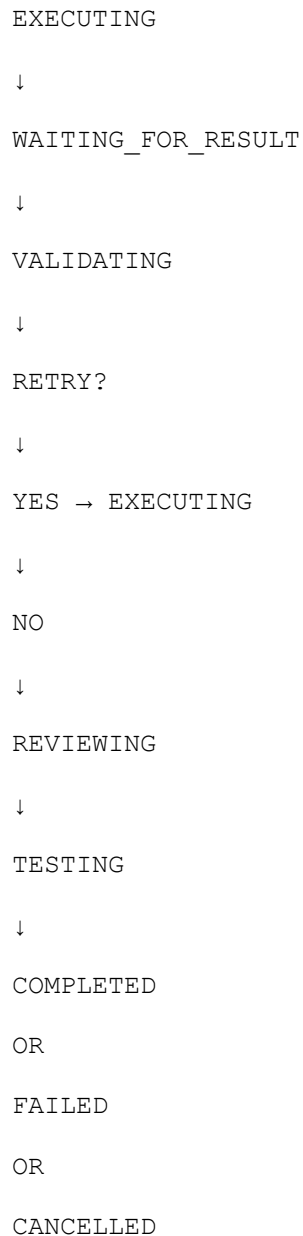
↓

PLANNING

↓

WAITING_FOR_TOOLS

↓



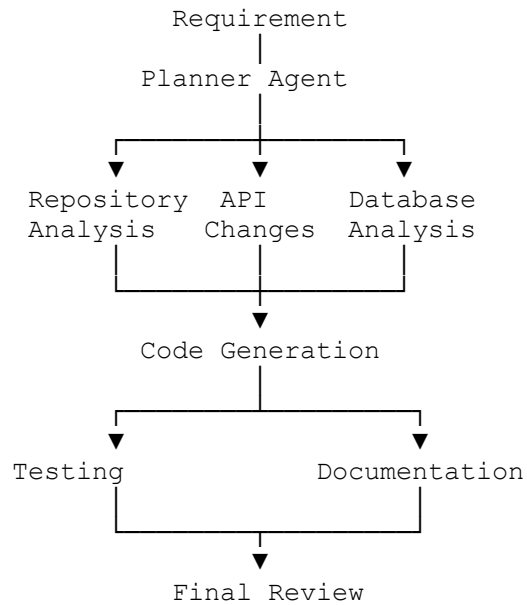
Each execution is persisted.

Therefore, if the system crashes, execution resumes from the last checkpoint.

5.7 Task Graph (DAG)

Instead of sequential execution, SDAP models work as a Directed Acyclic Graph (DAG).

Example:



This allows independent tasks to execute in parallel.

5.8 Parallel Execution

The orchestrator identifies independent tasks.

Example:

Instead of

Repository Scan

↓

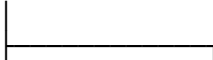
Unit Tests

↓

Documentation

Execute

Repository Scan



▼
Unit Tests

▼
Documentation

Result:

- Lower latency
- Better CPU utilization
- Faster completion

5.9 Context Management

Every agent receives only the context it requires.

Example:

Planner

Receives:

Requirements

Architecture

Previous Decisions

Not:

Entire Repository

Explorer

Receives:

Repository

Dependency Graph

File Structure

Reviewer

Receives:

Generated Diff

Coding Standards

Architecture

Not:

Terminal Logs

This minimizes token usage and improves reasoning quality.

5.10 Event Bus

Agents communicate through events rather than direct method calls.

Example Events

TASK_CREATED

PLAN_READY

REPOSITORY_INDEXED

FILE_MODIFIED

CODE_GENERATED

TEST_STARTED

TEST_COMPLETED

REVIEW_FAILED

DOCUMENTATION_READY

PR_CREATED

DEPLOYMENT_APPROVED

Benefits:

Loose coupling

Scalability

Fault isolation

Replayability

Auditability

5.11 Human Approval Workflow

Certain actions require approval.

Examples:

Delete repository

Force push

Production deployment

Database migration

Secrets modification

Infrastructure changes

Workflow:

Agent

↓

Approval Request

↓

User

↓

Approve?

YES

↓

Continue

NO

↓

Abort

5.12 Retry & Recovery Strategy

Failures are categorized.

Failure	Strategy
MCP timeout	Retry with exponential backoff
Tool unavailable	Alternate tool/server
LLM failure	Switch provider
Validation error	Ask agent to self-correct
Build failure	Return to Coding Agent
Test failure	Generate fix → re-test
Authentication failure	Refresh credentials

5.13 Memory Synchronization

Each execution updates:

- Conversation Memory
- Workspace Memory
- Repository Memory
- Artifact Store
- Knowledge Base

Memory synchronization occurs only after successful validation to avoid storing incorrect intermediate states.

5.14 Cost & Budget Control

The runtime continuously tracks:

- Prompt tokens
- Completion tokens
- Tool execution time

- LLM cost
- MCP calls
- Retry count

Policies may stop execution when configurable thresholds are exceeded.

5.15 Observability

Every orchestration step emits telemetry:

- Execution ID
- Agent ID
- Task ID
- Parent Task
- Duration
- Tool Used
- Tokens Consumed
- Cost
- Errors
- Retry Count

These metrics feed OpenTelemetry traces and Prometheus dashboards.

5.16 Architectural Decision Records (ADR)

ADR	Decision	Rationale
ADR-009 DAG-based execution		Enables parallelism and dependency management
ADR-010 Event-driven orchestration		Loose coupling and resilience
ADR-011 Stateful workflow engine		Supports long-running engineering tasks
ADR-012 Context partitioning		Reduces token usage and improves reasoning
ADR-013 Human approval checkpoints		Ensures governance for sensitive operations

5.17 Chapter Summary

The Agent Runtime and Orchestration layer transforms SDAP from a simple conversational AI into a **production-grade autonomous software engineering platform**. By combining planning, state management, event-driven workflows, context partitioning, DAG execution, and

governance, it provides the operational foundation required for reliable enterprise-scale AI-assisted development.

Chapter 6 – Agent Architecture & Deep Agent Design

6.1 Introduction

The Splash Developer Agent Platform (SDAP) adopts a **hierarchical multi-agent architecture** where autonomous, specialized agents collaborate under the supervision of an orchestration engine to accomplish complex software engineering tasks.

Unlike traditional AI assistants that rely on a single monolithic prompt, SDAP decomposes engineering work into specialized responsibilities. Each agent possesses:

- Dedicated objective
- Specialized reasoning
- Domain-specific skills
- Restricted tool access
- Independent execution state
- Memory isolation
- Security policies
- Quality validation

This architecture improves reasoning quality, reduces hallucinations, enables parallel execution, and provides clear operational boundaries.

6.2 Why Multi-Agent?

Traditional AI systems:

User

↓

LLM

↓

Answer

Problems:

✗ Huge prompts

✗ Context explosion

✗ Hallucinations

✗ No specialization

✗ Difficult debugging

✗ Poor scalability

SDAP

User

↓

Planner

↓

Explorer

↓

Coder

↓

Reviewer

↓

Tester

↓

Documentation

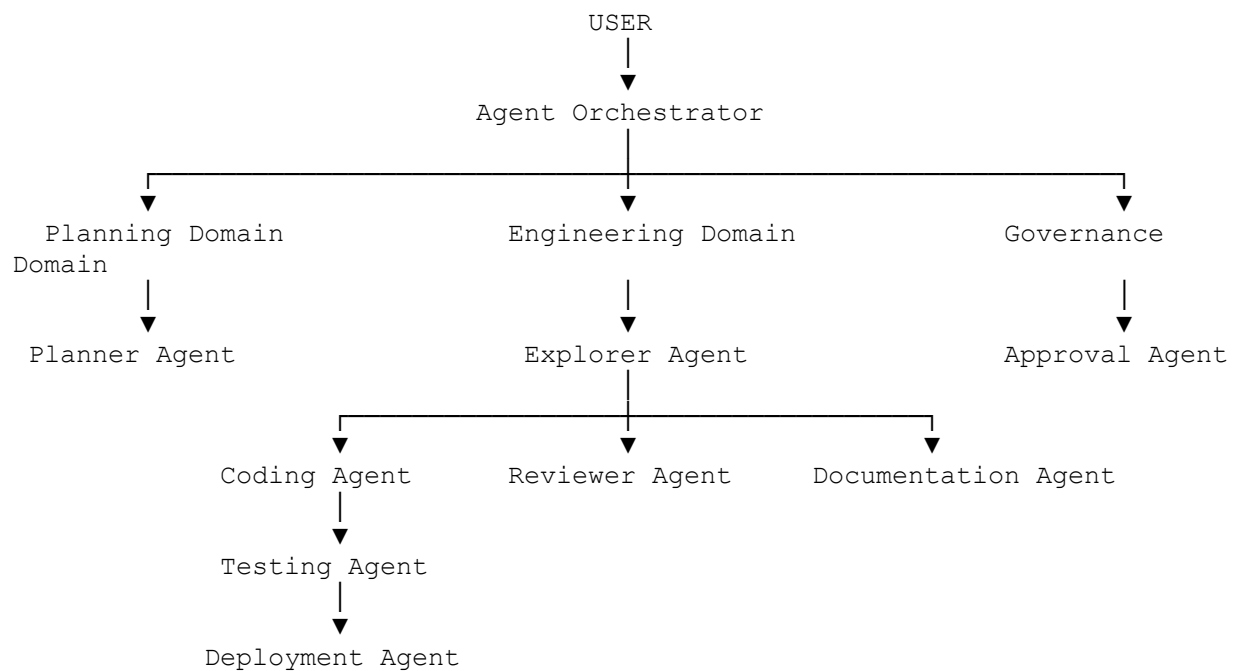
↓

Deployment

Advantages

- ✓ Modular
 - ✓ Testable
 - ✓ Scalable
 - ✓ Parallel
 - ✓ Easier Maintenance
 - ✓ Enterprise Ready
-

6.3 Complete Agent Hierarchy



6.4 Common Agent Interface

Every agent implements the same contract.

```
class BaseAgent:

    async def initialize()

    async def load_memory()

    async def load_skills()

    async def plan()

    async def execute()

    async def reflect()

    async def validate()

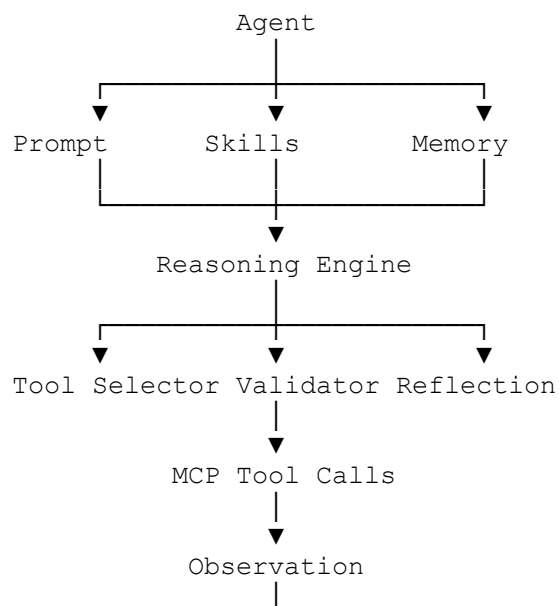
    async def finish()
```

Benefits

- Standard lifecycle
- Plug-and-play agents
- Easier testing
- Common telemetry
- Shared orchestration

6.5 Agent Internal Architecture

Every agent internally consists of multiple subsystems.



6.6 Planner Agent

Purpose

The Planner Agent converts user intent into a structured execution plan.

It **never writes code**.

Responsibilities

- Requirement analysis
- Task decomposition
- Dependency analysis
- Risk assessment
- Milestone creation
- Execution ordering
- Cost estimation

Input

Implement OAuth Login using Google Authentication

Output

Task 1

Repository Exploration

↓

Task 2

Identify Authentication Module

↓

Task 3

Design OAuth Flow

↓

Task 4

Implementation

↓

Task 5

Testing

↓

Task 6

Documentation

Tools

- Repository Index
- Project Metadata
- Memory
- Architecture Rules

No compiler access.

No Git access.

6.7 Explorer Agent

Purpose

Repository understanding.

Responsibilities

- File discovery
- API discovery
- Class discovery
- Dependency graph
- Symbol indexing
- Configuration discovery
- Framework detection

Tools

Filesystem MCP

Git MCP

Search MCP

Vector Search

Tree Sitter

AST Parser

Outputs

Repository Graph

Dependency Graph

Architecture Summary

6.8 Coding Agent

Purpose

Generate production-quality code.

Responsibilities

Feature Development

Bug Fixing

Refactoring

API Generation

Migration Scripts

Configuration

Documentation Comments

Allowed Tools

Filesystem MCP

Git MCP

Python MCP

Java MCP

Node MCP

Docker MCP

Compiler MCP

Forbidden

Database Delete

Production Deployment

Secret Manager

Execution Flow

Requirement

↓

Reason

↓

Choose Tool

↓

Generate Code

↓

Compile

↓

Observe

↓

Improve

↓

Repeat

↓

Success

6.9 Reviewer Agent

Purpose

Ensure quality.

Checks

Architecture

Naming

Cyclomatic Complexity

Security

Code Smells

Duplication

Performance

Standards

Produces

Review Score

Improvement Suggestions

Risk Report

6.10 Testing Agent

Responsibilities

Generate

Unit Tests

Integration Tests

API Tests

Regression Tests

Execute

pytest

JUnit

Jest

Playwright

Coverage

Outputs

Coverage Report

Failure Report

Regression Analysis

6.11 Documentation Agent

Produces

README

Architecture

API Docs

Release Notes

Sequence Diagrams

Mermaid Diagrams

Developer Guide

Deployment Guide

6.12 Deployment Agent

Responsible for

Dockerfile

Helm

Terraform

GitHub Actions

Azure DevOps

Jenkins

Kubernetes

Never deploys without approval.

6.13 Reflection Engine

Every agent performs self-reflection.

Reason

↓

Tool

↓

Observe

↓

Did it work?

↓

No

↓

Why?

↓

Improve

↓

Repeat

Reflection Questions

Did tool succeed?

Is answer complete?

Any architecture violations?

Any security issues?

Need another tool?

6.14 Inter-Agent Communication

Agents communicate via structured events.

Example

```
{
  "event": "CODE_GENERATED",
  "taskId": "T-142",
  "repository": "credit-api",
  "files": [
    "auth.py",
    "routes.py"
  ]
}
```

Other events:

PLAN_CREATED

REPOSITORY_SCANNED

TEST_FAILED

REVIEW_COMPLETED

DEPLOYMENT_READY

6.15 Agent Permissions Matrix

Agent	Read Repo	Modify Code	Run Tests	Git	Deploy
Planner	✓	✗	✗	✗	✗
Explorer	✓	✗	✗	✗	✗
Coding	✓	✓	Limited	✓	✗
Reviewer	✓	Comment only	✓	✗	✗
Tester	✓	✗	✓	✗	✗
Documentation	✓	Docs only	✗	✓	✗
Deployment	✓	✗	Verify	✓	Approval Required

6.16 Agent State

Each agent maintains:

- Current task
- Context window
- Loaded skills
- Tool session IDs
- MCP connections
- Execution history
- Retry count
- Cost metrics

This state is checkpointed so execution can resume after failures.

6.17 Agent Comparison

Capability	Planner	Explorer	Coder	Reviewer	Tester	Docs	Deploy
Planning	✓	✗	✗	✗	✗	✗	✗
Repository Analysis	Limited	✓	✓	✓	✓	✓	Limited
Code Generation	✗	✗	✓	✗	Test Code	Docs	Scripts
Reflection	✓	✓	✓	✓	✓	✓	✓
Tool Execution	Limited	✓	✓	✓	✓	Limited	✓

6.18 Mapping to Existing Industry Systems

Platform	Similar Concept	SDAP Enhancement
Cursor	Composer/Agent	Dedicated Planner + Workflow Engine
Claude Code	Coding Agent	Adds persistent orchestration, skills, and governance
OpenAI Codex	Code Generation	Multi-agent collaboration with enterprise tools
Devin	Autonomous Engineer	Open, MCP-native architecture with approval workflows
OpenHands	Multi-step coding	Enterprise security, memory, and observability
Salesforce Agentforce	Agent orchestration	Deeper developer-centric toolchain and language-specific skills

6.19 Key Architectural Decisions (ADR)

ADR	Decision	Rationale
ADR-014	Specialized agents	Better reasoning and maintainability
ADR-015	Shared base interface	Consistent lifecycle and telemetry
ADR-016	Reflection before completion	Improves quality and reduces hallucinations
ADR-017	Principle of least privilege	Security through restricted tool access
ADR-018	Event-based communication	Decoupled, scalable coordination

Chapter Summary

The Agent Architecture defines the **organizational structure** of the SDAP intelligence layer. Each agent has a narrowly scoped responsibility, explicit permissions, dedicated skills, and standardized lifecycle. Together, they form a coordinated engineering team capable of executing complex software development tasks while remaining secure, observable, and extensible.

Chapter 7 – Enterprise Skills Framework & Skill Registry

7.1 Introduction

The **Skills Framework** is the intelligence layer of SDAP.

While Large Language Models provide reasoning capabilities, enterprise software engineering requires repeatable expertise that extends beyond natural language understanding.

A Skill encapsulates organizational knowledge, engineering standards, reusable workflows, validation rules, templates, tool mappings, and execution strategies into modular, reusable components.

Skills allow the platform to consistently apply best practices regardless of the underlying LLM.

7.2 Why Skills?

Without Skills:

User

↓

LLM

↓

Generate Code

Problems:

- Depends entirely on prompt quality
 - No consistency
 - No coding standards
 - No organizational knowledge
 - Difficult to maintain
 - Prompt duplication
 - No version control
-

With Skills:

User

↓

Planner

↓

Load Skills

↓

Reason

↓

Execute Tools

↓

Validate

↓

Response

Benefits:

- ✓ Consistent behavior
- ✓ Organization-specific knowledge
- ✓ Reusable engineering expertise
- ✓ Easy updates

- ✓ Independent versioning
 - ✓ Better reasoning
 - ✓ Lower token usage
 - ✓ Enterprise governance
-

7.3 What is a Skill?

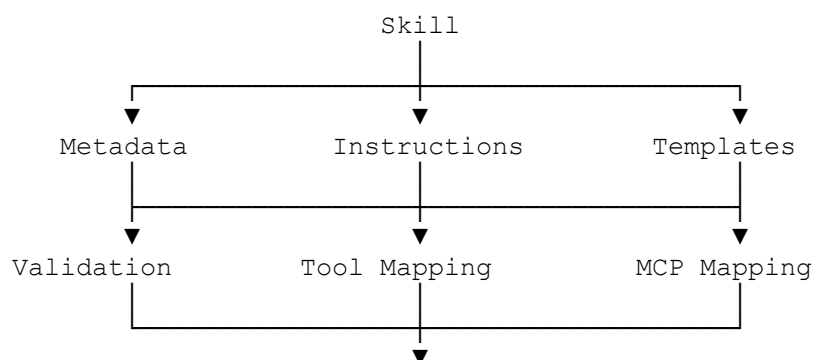
A Skill is a **self-contained capability package** that teaches an agent how to solve a specific class of problems.

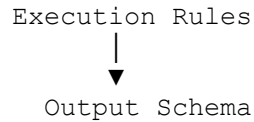
A Skill is **not** a prompt.

It is a structured asset containing:

- Metadata
 - Instructions
 - Workflows
 - Templates
 - Policies
 - Validation Rules
 - Tool Requirements
 - MCP Dependencies
 - Examples
 - Output Contracts
-

7.4 Enterprise Skill Architecture





7.5 Skill Components

Every Skill contains the following sections:

Component	Description
Metadata	Name, version, owner
Purpose	Business objective
Instructions	Engineering guidance
Tool Mapping	Required tools
MCP Mapping	Required MCP servers
Validation	Success criteria
Templates	Code templates
Examples	Few-shot examples
Output Schema	Expected output format
Dependencies	Other skills
Security Policy	Allowed operations
Version	Semantic version

7.6 Skill Metadata Schema

Example:

```
id: python-fastapi-api
name: FastAPI API Development
version: 2.1.0
author: Splash AI Team
category: Backend
language: Python
framework: FastAPI
dependencies:
  - python-core
```

- postgres
- testing

required_tools:

- filesystem
- git
- python
- docker

required_mcps:

- filesystem-mcp
- python-mcp
- docker-mcp

approval_required: false

supports_parallel_execution: true

7.7 Skill Lifecycle

Every Skill progresses through defined lifecycle stages.

Draft

↓

Review

↓

Approved

↓

Published

↓

Loaded

↓

Executing

↓

Deprecated

↓

Archived

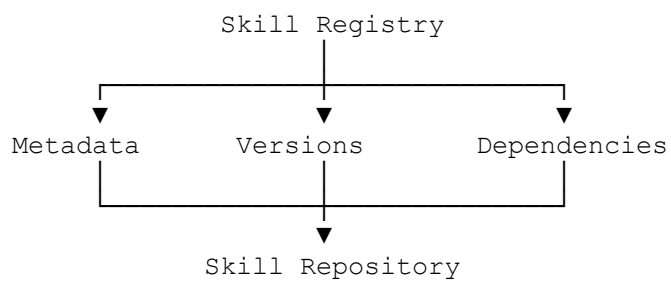
7.8 Skill Registry

The Skill Registry is the central catalog that stores and manages all available Skills.

Responsibilities:

- Registration
- Discovery
- Search
- Versioning
- Dependency Resolution
- Access Control
- Usage Analytics
- Deprecation

Architecture:



7.9 Skill Loader

When an agent receives a task:

Task

↓

Planner

↓

Skill Loader

↓

Resolve Dependencies

↓

Download Skills

↓

Initialize

↓

Inject into Context

↓

Execution

The Skill Loader ensures only the necessary Skills are loaded, minimizing token usage and startup time.

7.10 Skill Discovery

Agents do **not** hardcode Skill names.

Instead, they query the Skill Registry based on task metadata.

Example:

User:

"Build OAuth Login"

Skill Registry Returns:

OAuth Skill

↓

FastAPI Skill

↓

JWT Skill

↓

PostgreSQL Skill

↓

Testing Skill

7.11 Skill Dependency Graph

Skills can depend on other Skills.

FastAPI Skill

|

├─ Python Core Skill

├─ Async Programming Skill

├─ PostgreSQL Skill

├─ JWT Skill

└─ Testing Skill

The Skill Loader resolves the dependency graph before execution.

7.12 Skill Categories

Language Skills

- Python
- Java
- Go
- Rust
- JavaScript
- TypeScript
- C#
- Kotlin

Framework Skills

- FastAPI
- Flask
- Spring Boot
- React
- Angular

- Vue
 - Express
 - Django
-

Database Skills

- PostgreSQL
 - MongoDB
 - Redis
 - MySQL
 - Oracle
-

DevOps Skills

- Docker
 - Kubernetes
 - Helm
 - Terraform
 - GitHub Actions
-

Testing Skills

- pytest
 - JUnit
 - Jest
 - Playwright
 - Cypress
-

Cloud Skills

- AWS
 - Azure
 - GCP
-

Security Skills

- OAuth
 - JWT
 - RBAC
 - Encryption
 - Secrets Management
-

AI Skills

- Prompt Engineering
 - RAG
 - Vector Search
 - Agent Orchestration
 - MCP Integration
-

7.13 Skill Package Structure

python-fastapi-api/

├─ skill.yaml

├─ instructions.md

├─ workflow.yaml

├─ templates/

| └─ api.py

| └─ router.py

| └─ service.py

├─ validation/

| └─ rules.yaml

├─ examples/

├─ prompts/

├─ tests/

└─ version.txt

7.14 Skill Execution Flow

Agent

↓

Select Skill

↓

Load Metadata

↓

Load Instructions

↓

Load Templates

↓

Load Tool Mapping

↓

Load MCP Mapping

↓

Execute

↓

Validate

↓

Return Result

7.15 Skill-to-Tool Mapping

Skill	Required Tools
Python	Filesystem, Python, Git
React	Filesystem, Node, Browser
PostgreSQL	Database, SQL
Docker	Docker Engine

Skill	Required Tools
Kubernetes	kubect!, Helm
OAuth	Browser, Secrets Manager

This mapping keeps Skills independent of specific tool implementations.

7.16 Skill-to-MCP Mapping

Skill	MCP Servers
Python	Python MCP, Filesystem MCP
Java	Java MCP
React	Node MCP, Browser MCP
Git	Git MCP
PostgreSQL	PostgreSQL MCP
Docker	Docker MCP
Kubernetes	Kubernetes MCP
Browser Automation	Playwright MCP

7.17 Skill Versioning

Skills follow **Semantic Versioning (SemVer)**:

- **Major:** Breaking changes (e.g., incompatible workflow changes)
- **Minor:** New features (e.g., support for a new framework)
- **Patch:** Bug fixes and documentation updates

The Skill Registry supports:

- Version pinning per workspace
 - Backward compatibility checks
 - Upgrade recommendations
 - Rollback to previous versions
-

7.18 Enterprise Skill Marketplace

Although SDAP will initially ship with built-in Skills, the architecture supports an internal marketplace where teams can publish and share reusable Skills.

Capabilities:

- Publish new Skills
- Private or organization-wide visibility
- Approval workflows
- Digital signing
- Dependency validation
- Usage metrics
- Ratings and feedback (optional)

7.19 Security & Governance

Every Skill is governed by policy.

A Skill definition includes:

- Allowed tools
- Allowed MCP servers
- Required permissions
- Approval requirements
- Maximum execution time
- Data access scope

Example:

- A **Deployment Skill** may require manager approval.
- A **Filesystem Skill** may only access the current workspace.
- A **Database Skill** may be read-only in production environments.

7.20 Key Architectural Decisions (ADR)

ADR	Decision	Rationale
ADR-019	Skills are modular packages	Reusability and maintainability
ADR-020	Dynamic skill loading	Reduce memory footprint and token usage
ADR-021	Skills are versioned	Controlled evolution and rollback
ADR-022	Skills are LLM-independent	Avoid vendor lock-in

ADR	Decision	Rationale
ADR-023	Skills declare tool and MCP dependencies	Clear execution contracts

7.21 Chapter Summary

The Skills Framework is the **knowledge distribution mechanism** of SDAP. By separating reusable engineering expertise from agent logic, it enables consistent, governable, and extensible behavior across multiple languages, frameworks, and enterprise environments.

Chapter 8 — Model Context Protocol (MCP) Architecture & Enterprise Integration

Version: 1.0

8.1 Introduction

The **Model Context Protocol (MCP)** is the standardized communication protocol between SDAP and external systems.

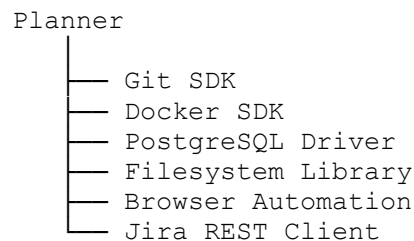
Rather than embedding tool-specific logic inside AI agents, MCP provides a uniform, discoverable, secure, and extensible interface for exposing tools, resources, and prompts to the agent ecosystem.

This architecture allows SDAP to integrate with development environments, enterprise systems, cloud platforms, databases, browsers, and language runtimes without requiring changes to agent implementations.

Key Design Principle: Agents never directly interact with Git, Docker, PostgreSQL, or the filesystem. They only interact with the MCP Client. This abstraction ensures portability, governance, and easier extensibility.

8.2 Why MCP?

Without MCP:

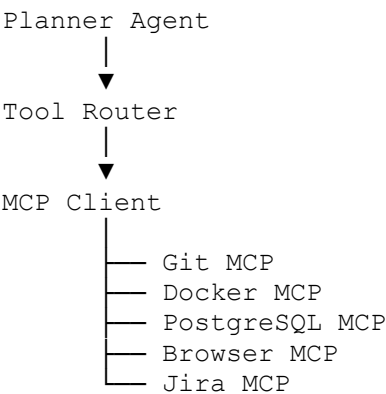


Problems:

- Tight coupling
- Vendor lock-in
- Multiple authentication mechanisms

- Duplicate implementations
- Hard to test
- Difficult upgrades

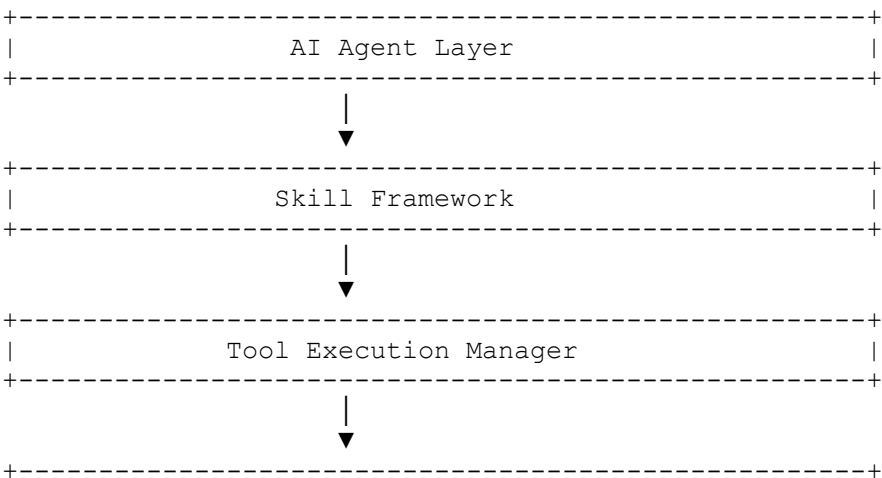
With MCP:

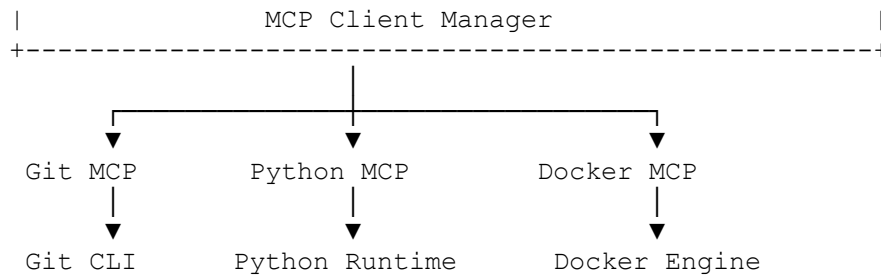


Benefits:

- Standard protocol
- Dynamic discovery
- Centralized authentication
- Version compatibility
- Pluggable integrations
- Consistent telemetry

8.3 MCP Architectural Position





The **MCP Client Manager** becomes the single gateway to every external capability.

8.4 MCP Core Concepts

SDAP adopts the official MCP concepts and extends them with enterprise governance.

MCP Concept	SDAP Interpretation
Tool	Executable capability (e.g., run tests, commit code)
Resource	Read-only context (e.g., source files, logs, schemas)
Prompt	Reusable interaction template for agents
Client	Component that discovers and invokes capabilities
Server	Provider exposing capabilities
Transport	Communication channel (STDIO, Streamable HTTP, SSE)
Session	Authenticated interaction lifecycle

8.5 Enterprise MCP Components

1. MCP Client Manager

Responsibilities:

- Server discovery
 - Session management
 - Authentication
 - Capability caching
 - Load balancing
 - Retry policies
 - Parallel execution
 - Telemetry
-

2. MCP Registry

Stores:

- Registered servers
 - Version metadata
 - Supported transports
 - Authentication methods
 - Health status
 - Workspace visibility
 - Tags (Language, Database, DevOps, etc.)
-

3. Capability Catalog

Rather than hardcoding tool names, SDAP queries a catalog.

Example:

```
server: python-mcp

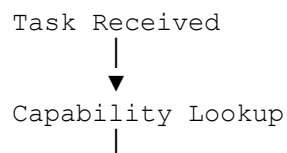
tools:
  - run_python
  - install_package
  - lint
  - format
  - pytest

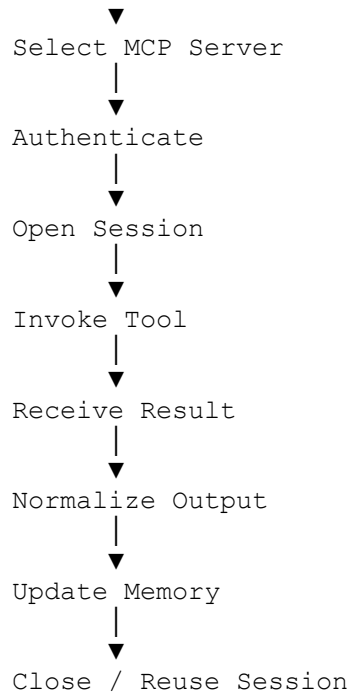
resources:
  - virtualenv
  - requirements.txt

prompts:
  - optimize_python
  - explain_traceback
```

Agents discover capabilities dynamically.

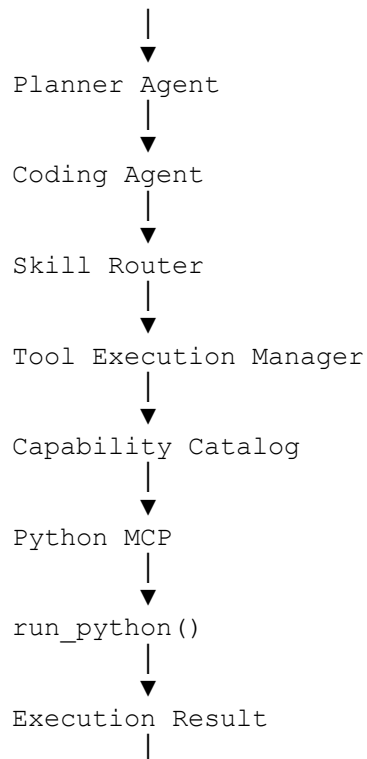
8.6 MCP Client Lifecycle

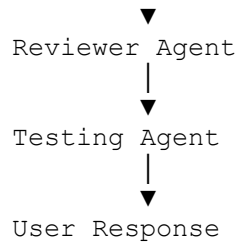




8.7 Enterprise Tool Invocation Flow

User:
"Add OAuth login"





8.8 MCP Message Flow (JSON-RPC)

MCP communication is based on JSON-RPC.

Example request:

```
{
  "jsonrpc": "2.0",
  "id": "req-1024",
  "method": "tools/call",
  "params": {
    "tool": "run_python",
    "arguments": {
      "file": "main.py"
    }
  }
}
```

Example response:

```
{
  "jsonrpc": "2.0",
  "id": "req-1024",
  "result": {
    "status": "success",
    "stdout": "Application started.",
    "stderr": ""
  }
}
```

The Tool Execution Manager normalizes responses so agents interact with a consistent internal model regardless of the underlying MCP server.

8.9 Transport Layer Strategy

SDAP supports multiple MCP transports:

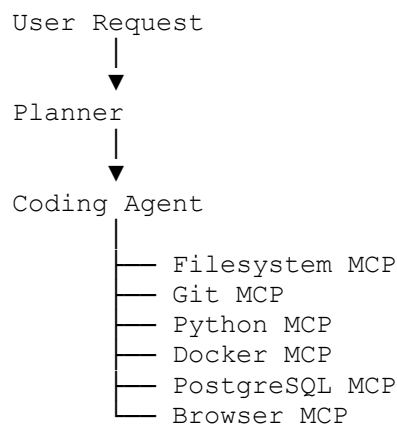
Transport	Use Case	Recommended
STDIO	Local development tools	✓ Local IDE integrations
Streamable HTTP	Enterprise microservices	✓ Production default
SSE	Long-running streams	✓ Live logs and progress
WebSocket (<i>planned extension</i>)	Interactive sessions	Future enhancement

Transport selection is configurable per server.

8.10 Multi-MCP Orchestration

Complex tasks often require multiple MCP servers.

Example:



The Tool Execution Manager coordinates dependencies, parallelism, and retries across all participating servers.

8.11 Enterprise MCP Server Taxonomy

Category	Representative Servers
Source Control	Git, GitHub, GitLab
Language Runtime	Python, Java, Node.js, Go, Rust
Build & Package	Maven, Gradle, npm, pip
Databases	PostgreSQL, MySQL, MongoDB, Redis
DevOps	Docker, Kubernetes, Helm, Terraform

Category	Representative Servers
Browser	Playwright, Chromium
Cloud	AWS, Azure, GCP
Collaboration	Jira, Confluence, Slack, Teams
Observability	Prometheus, Grafana, OpenTelemetry

Each category will be documented with:

- Architecture
- Authentication
- Tool catalog
- Resources
- Example requests
- Failure scenarios
- Deployment guidance

8.12 Security Model

Security is enforced at multiple levels:

1. **Authentication**
 - OAuth2
 - API Keys
 - mTLS (optional)
2. **Authorization**
 - RBAC
 - Workspace isolation
 - Tool-level permissions
3. **Execution Policies**
 - Allowlists
 - Read-only modes
 - Approval-required operations
4. **Audit**
 - Every tool call
 - Every resource access
 - Every prompt invocation

8.13 Error Handling & Recovery

Failures are categorized:

Failure	Recovery
Network timeout	Exponential backoff
Server unavailable	Failover to secondary server
Authentication expired	Token refresh
Tool not found	Capability rediscovery
Invalid arguments	Validation + retry
Partial workflow failure	Resume from checkpoint

8.14 Observability

Every MCP interaction emits:

- Request ID
- Session ID
- Workspace ID
- Tool Name
- Server Name
- Latency
- Payload Size
- Cost (if applicable)
- Success/Failure
- Retry Count

Integrated with:

- OpenTelemetry
 - Prometheus
 - Grafana
 - Centralized Audit Logs
-

8.15 Enterprise Architectural Decisions (ADR)

ADR	Decision	Rationale
ADR-024	MCP-first integration strategy	Standardize external capabilities

ADR	Decision	Rationale
ADR-025	Centralized MCP Client Manager	Simplify orchestration and governance
ADR-026	Capability Catalog	Dynamic discovery without hardcoding
ADR-027	Streamable HTTP as production default	Cloud-native scalability
ADR-028	Multi-server orchestration	Enable complex workflows spanning multiple tools

Chapter Summary

The MCP Architecture establishes the **integration backbone** of SDAP. By treating external capabilities as discoverable services rather than embedded libraries, the platform achieves modularity, security, and long-term extensibility. This design allows new tools, languages, and enterprise systems to be onboarded with minimal impact on the agent layer.

Chapter 9 – Enterprise Tool Framework & Tool Execution Engine

Version: 1.0

9.1 Introduction

The **Tool Execution Engine (TEE)** is the execution backbone of SDAP.

While agents perform reasoning and Skills provide domain expertise, neither directly executes external operations. Instead, they delegate all interactions to the Tool Execution Engine, which is responsible for securely invoking tools, managing execution lifecycles, enforcing policies, and returning normalized results.

This separation ensures that:

- Agents remain focused on reasoning.
- Skills remain declarative.
- Tools remain interchangeable.
- MCP servers remain implementation details.

The TEE acts as the **operating system** for external execution.

9.2 Why a Tool Execution Engine?

Without a dedicated execution layer:

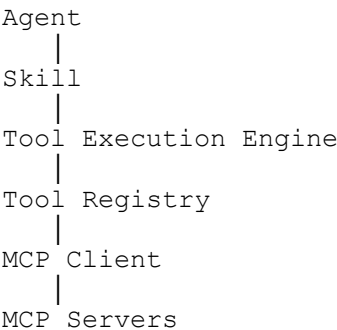
```
Agent
├── Git SDK
├── Docker SDK
├── Python Runtime
├── PostgreSQL Driver
├── Browser
└── kubectl
```

Problems:

- Tight coupling
- Duplicate authentication
- Inconsistent retries

- No governance
- Difficult testing
- Vendor-specific logic

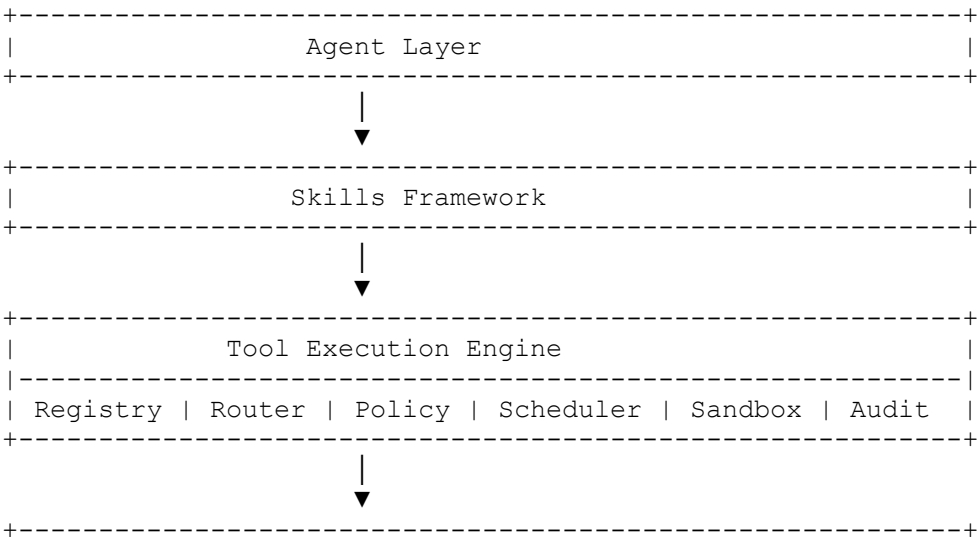
With the Tool Execution Engine:

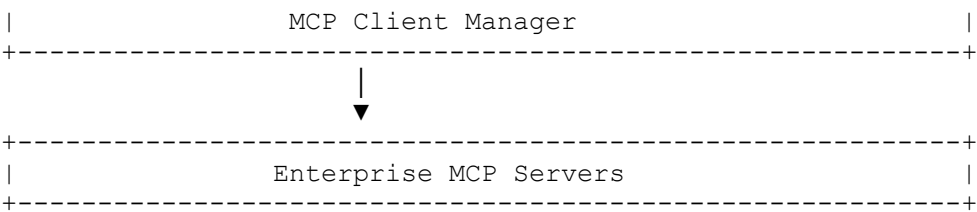


Benefits:

- Loose coupling
- Central governance
- Unified execution
- Standard contracts
- Better observability
- Easy extensibility

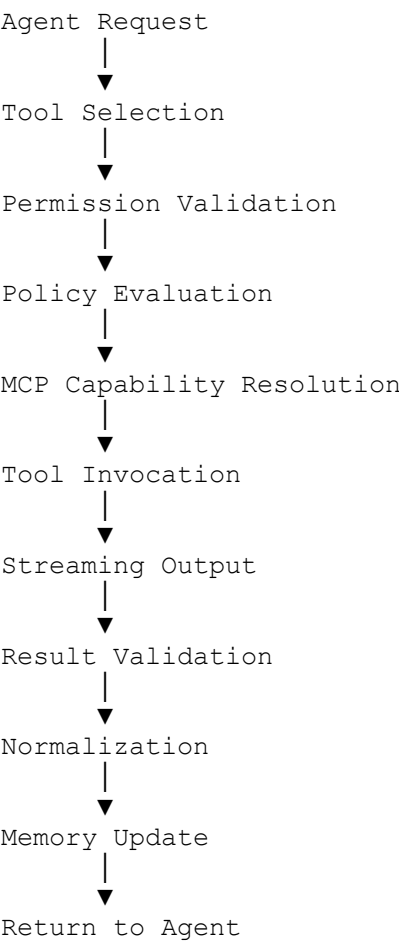
9.3 Architectural Position





9.4 Tool Execution Pipeline

Every tool invocation follows a standardized pipeline:



9.5 Core Components

1. Tool Registry

The Tool Registry is the authoritative inventory of all executable tools.

Responsibilities:

- Tool registration
- Metadata management
- Versioning
- Capability discovery
- Permission mapping
- Health monitoring

Example metadata:

```
tool_id: git.commit
display_name: Git Commit

category: Source Control

provider: git-mcp

version: 1.2.0

permissions:
  - repository.write

approval_required: false

timeout: 30s

retryable: true
```

2. Tool Router

The Tool Router determines the best execution path.

Selection criteria:

- Tool availability
- Server health
- Workspace policies
- Latency
- Cost
- User preferences

Example:

Create Docker Image

↓

Docker MCP Available?

↓

YES

↓

Route to Docker MCP

↓

NO

↓

Fallback Container MCP

3. Policy Engine

Every execution is validated against organizational policies.

Policies may include:

- Allowed workspaces
- Approved repositories
- Read-only mode
- Production restrictions
- Cost limits
- Time limits

Example:

```
policy:  
  workspace: production
```

```
allowed_tools:  
  - git.read  
  - postgres.read
```

```
blocked_tools:  
  - filesystem.delete  
  - docker.remove
```

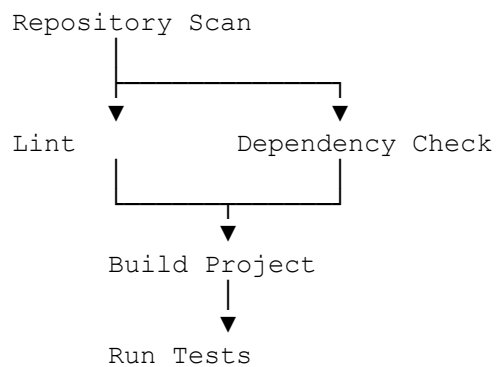
4. Execution Scheduler

The Scheduler optimizes execution by:

- Ordering tasks

- Running independent tools in parallel
- Managing priorities
- Enforcing concurrency limits
- Supporting cancellation
- Handling retries

Example DAG:



5. Sandbox Manager

All executable tools run within isolated environments.

Supported sandboxes:

- Docker containers
- Kubernetes pods
- Firecracker microVMs (future)
- Read-only filesystem mode
- Resource quotas

Controls:

- CPU
- Memory
- Disk
- Network
- Execution time

6. Result Normalizer

Different tools produce different outputs.

The Result Normalizer converts them into a consistent internal format.

Example:

```
{
  "status": "SUCCESS",
  "tool": "pytest",
  "exitCode": 0,
  "summary": "128 tests passed",
  "artifacts": [
    "coverage.xml",
    "report.html"
  ]
}
```

Agents consume only this normalized schema.

9.6 Tool Categories

Source Control

- Git
 - GitHub
 - GitLab
-

Filesystem

- Read
 - Write
 - Move
 - Search
 - Archive
-

Runtime

- Python
- Java
- Node.js
- Go
- Rust

Build

- Maven
- Gradle
- npm
- pip

Testing

- pytest
- JUnit
- Jest
- Playwright
- Cypress

DevOps

- Docker
- Kubernetes
- Helm
- Terraform

Database

- PostgreSQL
- MongoDB
- Redis
- MySQL

Browser

- Chromium
- Playwright

Enterprise

- Jira
 - Confluence
 - Slack
 - Teams
-

9.7 Tool Contract

Every tool implements a standard interface.

```
class Tool:

    async def initialize()

    async def authorize()

    async def validate()

    async def execute()

    async def stream()

    async def cleanup()

    async def health_check()
```

This allows any tool to be plugged into SDAP without changing agent code.

9.8 Tool Lifecycle

```
Discovered
|
Registered
|
Initialized
|
Authorized
|
Executing
|
Streaming
|
```

9.9 Parallel Execution

The engine supports concurrent tool invocation where dependencies allow.

Example:

```
Explorer Agent
├── Filesystem Search
├── Git History
├── Dependency Analysis
└── Package Inspection
```

These operations execute simultaneously, reducing overall latency.

9.10 Streaming Execution

Long-running tools (e.g., builds, tests) stream incremental output.

Example:

```
[00:01] Installing dependencies...
[00:05] Running unit tests...
[00:18] 120 tests passed.
[00:24] Coverage: 94%.
```

The UI subscribes via WebSockets or Server-Sent Events.

9.11 Security & Governance

The TEE enforces:

- Tool-level RBAC
- Workspace isolation
- Rate limits
- Approval workflows
- Audit logging

- Secret redaction
- Output sanitization

No tool executes without authorization.

9.12 Observability

Every execution emits telemetry:

Metric	Description
Tool ID	Invoked tool
Session ID	Execution session
Agent ID	Calling agent
Latency	Total execution time
Retry Count	Number of retries
Exit Code	Process result
Cost	Resource usage
Output Size	Bytes returned

Integrated with:

- OpenTelemetry
- Prometheus
- Grafana
- Loki

9.13 Failure Handling

Failure	Recovery
Tool timeout	Retry with backoff
MCP unavailable	Failover to alternate server
Invalid arguments	Validate and retry
Permission denied	Surface approval request
Sandbox failure	Restart isolated environment
Partial workflow	Resume from checkpoint

9.14 Architectural Decisions (ADR)

ADR	Decision	Rationale
ADR-029	Central Tool Execution Engine	Decouple reasoning from execution
ADR-030	Standard tool contract	Uniform integration model
ADR-031	Policy-first execution	Enterprise governance
ADR-032	Sandboxed execution	Security and isolation
ADR-033	Result normalization	Consistent agent experience

9.15 Chapter Summary

The Tool Execution Engine is the **execution operating system** of SDAP. It transforms high-level agent intent into secure, observable, and policy-compliant tool invocations while abstracting the complexity of MCP servers, runtime environments, and external systems.

Chapter 10 – Enterprise MCP Server Ecosystem & Language-Specific Integrations

Version: 1.0

10.1 Introduction

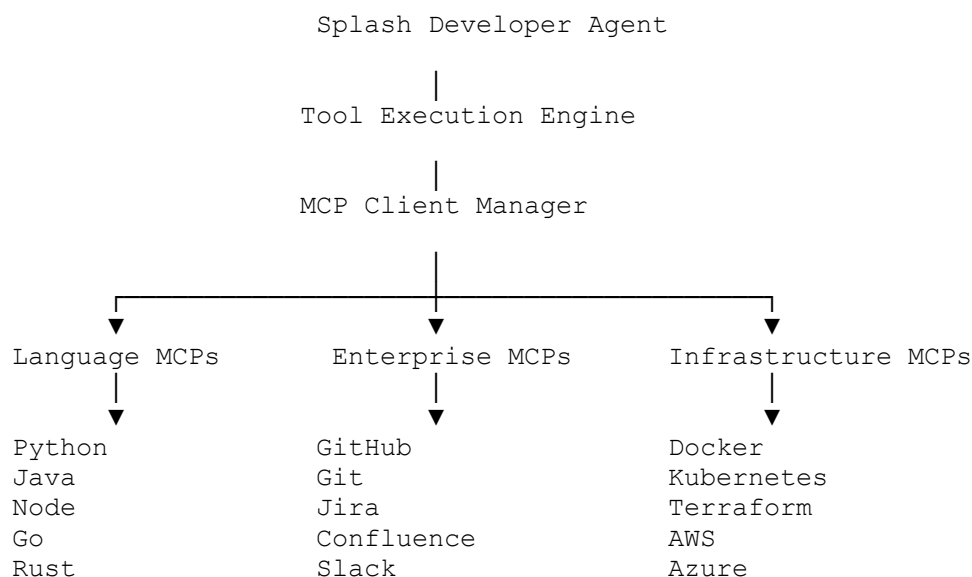
The **Model Context Protocol (MCP)** provides a standardized way for SDAP to interact with external systems.

Rather than tightly coupling the Developer Agent to specific SDKs or APIs, SDAP treats every external capability as an **MCP Server** exposing standardized:

- Tools
- Resources
- Prompts

This architecture allows new languages, frameworks, cloud providers, and enterprise systems to be integrated without changing the agent runtime.

10.2 Enterprise MCP Architecture



10.3 MCP Server Classification

We classify MCP servers into **six enterprise domains**.

Domain	Purpose	Examples
Language Runtime	Execute and analyze code	Python, Java, Go, Node
Source Control	Repository operations	Git, GitHub, GitLab
Infrastructure	Build and deployment	Docker, Kubernetes, Terraform
Database	Data access	PostgreSQL, MongoDB, Redis
Enterprise Productivity	Collaboration	Jira, Confluence, Slack
Cloud	Infrastructure APIs	AWS, Azure, GCP

10.4 Language Runtime MCP Servers

These servers provide language-aware capabilities beyond simply executing commands.

10.4.1 Python MCP Server

Purpose

Provide deep integration with Python projects, virtual environments, package managers, linters, formatters, test frameworks, and static analysis.

Capabilities

- Execute Python scripts
- Manage virtual environments
- Install packages
- Run pytest
- Format code (Black)
- Lint (Ruff/Flake8)
- Type checking (MyPy)
- AST analysis
- Dependency inspection
- Coverage generation

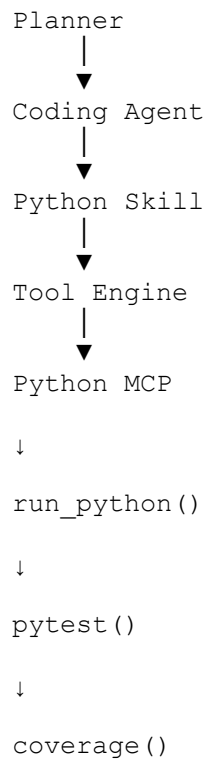
Exposed Tools

Tool	Description
<code>run_python</code>	Execute a Python file
<code>create_venv</code>	Create virtual environment
<code>install_package</code>	Install dependency
<code>pytest</code>	Execute test suite
<code>black_format</code>	Format source code
<code>ruff_lint</code>	Static analysis
<code>mypy_check</code>	Type validation
<code>generate_requirements</code>	Build requirements.txt

Resources

- `pyproject.toml`
- `requirements.txt`
- `.venv`
- Coverage reports
- Dependency graph

Example Workflow



Production Deployment

- Containerized
 - Read-only filesystem by default
 - Workspace isolation
 - Ephemeral virtual environments
 - Configurable package mirrors
-

10.4.2 Java MCP Server

Purpose

Support enterprise Java applications.

Supported Frameworks

- Spring Boot
- Quarkus
- Micronaut
- Jakarta EE

Tools

- Maven
- Gradle
- JUnit
- SpotBugs
- PMD
- Checkstyle
- JaCoCo

Capabilities

- Build projects
 - Run unit tests
 - Dependency tree
 - Generate JavaDocs
 - Static code analysis
 - Bytecode inspection
-

10.4.3 Node.js / React MCP Server

Framework Support

- React
- Next.js
- Express
- NestJS
- Vite

Tools

- npm
- yarn
- pnpm
- ESLint
- Prettier
- Jest
- Vitest
- Playwright

Capabilities

- Package management
 - Development server
 - Production build
 - Linting
 - Bundle analysis
 - Test execution
-

10.4.4 Go MCP Server

Capabilities

- go build
 - go test
 - go mod
 - staticcheck
 - gofmt
 - benchmarking
 - race detection
-

10.4.5 Rust MCP Server

Capabilities

- cargo build
- cargo test
- clippy
- rustfmt
- benchmarking

10.4.6 .NET MCP Server

Frameworks

- ASP.NET Core
- Blazor

Capabilities

- dotnet build
- dotnet test
- NuGet management
- Roslyn analysis

10.5 Source Control MCP Servers

Git MCP

Purpose

Repository operations.

Tools

clone

checkout

branch

merge

rebase

commit

diff

blame

tag

stash

Additional Enterprise Features

- Signed commits
- Protected branch awareness
- Conflict detection
- PR assistance

GitHub MCP

Capabilities

- Create Pull Requests
- Review PRs
- Issues
- Actions
- Releases
- Labels
- Discussions

Authentication

GitHub App

PAT

OAuth

GitLab MCP

Supports

- Merge Requests
 - Pipelines
 - CI/CD
 - Projects
 - Groups
 - Security Dashboard
-

10.6 Infrastructure MCP Servers

Docker MCP

Capabilities

- Build images
- Run containers
- Inspect images
- Logs
- Volumes
- Networks

Restrictions

Never remove production containers automatically.

Kubernetes MCP

Capabilities

- Apply manifests
- Describe resources
- Rollout status
- Logs
- Port forward
- Scale deployments

Approval Required

- Delete Namespace

- Production Rollout
 - Cluster Admin Operations
-

Terraform MCP

Capabilities

- Validate
 - Plan
 - Apply
 - Destroy (approval required)
 - Module inspection
-

10.7 Database MCP Servers

PostgreSQL MCP

Capabilities

- Execute SELECT
- Explain query
- Schema discovery
- Index analysis
- Migration generation

Restrictions

- Read-only in Production
 - DDL requires approval
-

MongoDB MCP

Capabilities

- Query collections
- Explain plans

- Aggregation
 - Schema inspection
-

Redis MCP

Capabilities

- Inspect keys
 - TTL
 - Cache analysis
 - Memory statistics
-

10.8 Browser Automation MCP

Playwright MCP

Capabilities

- Browser launch
- UI automation
- Screenshots
- Accessibility audit
- Performance metrics
- Network tracing

Example Flow

Testing Agent

↓

Playwright MCP

↓

Open Browser

↓

Execute UI Test

↓

Screenshot

↓

Generate Report

10.9 Enterprise Productivity MCP

Jira MCP

Capabilities

- Create issues
 - Update status
 - Add comments
 - Link pull requests
 - Sprint management
-

Confluence MCP

Capabilities

- Read pages
 - Update documentation
 - Publish architecture docs
 - Search knowledge base
-

Slack / Teams MCP

Capabilities

- Send notifications
- Approval workflows
- Incident alerts
- Engineering updates

10.10 Cloud MCP Servers

AWS MCP

Services

- S3
- Lambda
- ECS
- EKS
- RDS
- Secrets Manager
- CloudWatch

Azure MCP

Services

- App Service
- AKS
- Cosmos DB
- Azure SQL
- Key Vault

Google Cloud MCP

Services

- Cloud Run
 - GKE
 - BigQuery
 - Secret Manager
 - Cloud Storage
-

10.11 MCP Authentication Strategy

Server Type	Authentication
GitHub	GitHub App / OAuth
GitLab	OAuth / PAT
Jira	OAuth 2.0
PostgreSQL	Database credentials / IAM
Kubernetes	Service Account / kubeconfig
Docker	Local socket / TLS
AWS	IAM Role
Azure	Managed Identity
GCP	Service Account

Credentials are never exposed to agents. The MCP Client Manager retrieves secrets from Vault or a secrets manager.

10.12 High Availability & Scaling

To ensure enterprise reliability:

- Multiple MCP instances behind a load balancer
 - Health checks and automatic failover
 - Capability cache replication
 - Session affinity where required
 - Circuit breakers for unstable services
-

10.13 Custom MCP Server Development

SDAP supports internal MCP servers for proprietary systems.

Recommended project structure:

```
loan-origination-mcp/  
├── server.py  
├── tools/  
├── resources/  
├── prompts/  
└── auth/
```

```
|— config/
|— tests/
|— README.md
```

Development standards:

- Follow official MCP specification
- Expose typed tool contracts
- Implement health endpoints
- Emit OpenTelemetry traces
- Support OAuth or mTLS
- Provide version metadata

10.14 MCP Server Selection Strategy

The Tool Execution Engine selects servers based on:

1. Capability match
2. Workspace policy
3. Health status
4. Authentication availability
5. Latency
6. Cost
7. User preferences
8. Geographic locality

This allows dynamic routing and future multi-provider support.

10.15 Enterprise Recommendations for Splash Financial

Based on your stack (FastAPI backend, React frontend, PostgreSQL, Groq LLM, AI-first development), I recommend prioritizing these MCP servers for **Phase 1**:

Priority	MCP Server	Purpose
P0	Filesystem MCP	Repository operations
P0	Git MCP	Version control
P0	Python MCP	FastAPI backend development
P0	Node.js/React MCP	Frontend development

Priority	MCP Server	Purpose
P0	PostgreSQL MCP	Database exploration
P1	Docker MCP	Local container builds
P1	Playwright MCP	UI testing
P1	GitHub MCP	Pull requests & code review
P2	Kubernetes MCP	Deployment automation
P2	Jira MCP	Work item integration
P2	Confluence MCP	Documentation publishing
P3	AWS/Azure/GCP MCP	Cloud deployment (as needed)

This phased approach minimizes complexity while delivering immediate value to developers.

10.16 Architectural Decisions (ADR)

ADR	Decision	Rationale
ADR-034	Domain-specific MCP classification	Clear ownership and extensibility
ADR-035	Language-aware MCP servers	Deep ecosystem integration
ADR-036	Secretless agent model	Prevent credential exposure
ADR-037	HA MCP deployments	Enterprise reliability
ADR-038	Phased MCP adoption	Faster implementation with lower risk

Chapter Summary

The Enterprise MCP Ecosystem provides SDAP with a standardized, secure, and extensible integration layer for languages, development tools, infrastructure, databases, and enterprise platforms. By organizing MCP servers into well-defined domains and introducing governance, authentication, and high availability, SDAP can scale from local development to enterprise-wide deployment without changing its core agent architecture.

Chapter 11 – Backend Microservices & Infrastructure Design

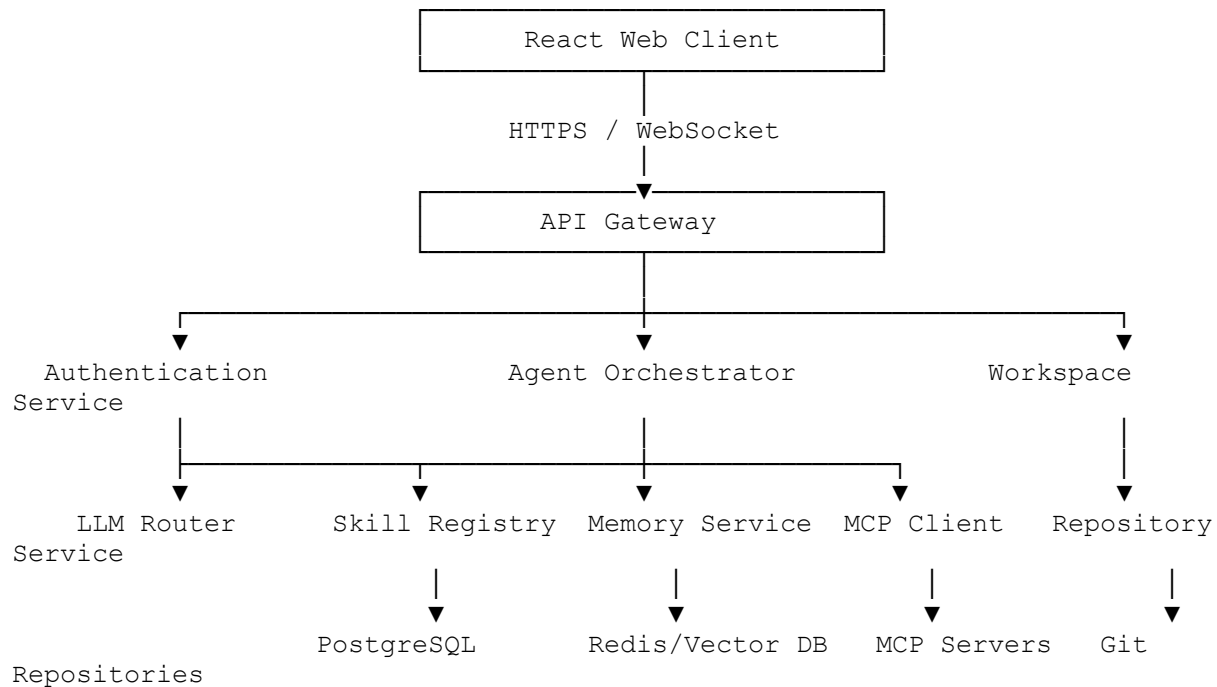
Version: 1.0

11.1 Introduction

The SDAP backend is designed using a **cloud-native microservices architecture** to support scalability, fault isolation, independent deployment, and future extensibility.

Each service owns a single bounded context and communicates through well-defined APIs and asynchronous events. This approach enables teams to evolve services independently while maintaining clear ownership and operational visibility.

11.2 High-Level Architecture



11.3 Service Decomposition

Service	Responsibility	Technology
API Gateway	Entry point, routing, rate limiting	FastAPI
Auth Service	Authentication & authorization	FastAPI + JWT/OAuth2
Agent Orchestrator	Workflow coordination	FastAPI
Planner Service	Task planning	FastAPI
LLM Router	Multi-model management	FastAPI
Skill Registry	Skill discovery & versioning	FastAPI
Memory Service	Context & persistence	FastAPI
Tool Execution Service	Tool orchestration	FastAPI
MCP Client Service	MCP communication	FastAPI
Repository Service	Git operations	FastAPI
Notification Service	WebSocket & events	FastAPI

11.4 API Gateway

The API Gateway is the only externally exposed backend component.

Responsibilities:

- Request routing
- JWT validation
- OAuth token verification
- Rate limiting
- Request logging
- API versioning
- CORS management
- WebSocket upgrades

Example routes:

Endpoint	Target Service
/api/v1/chat	Agent Orchestrator
/api/v1/workspaces	Workspace Service
/api/v1/skills	Skill Registry
/api/v1/tasks	Agent Orchestrator
/ws/events	Notification Service

11.5 Agent Orchestrator Service

The orchestrator is the central coordination service.

Responsibilities:

- Create execution plans
- Manage agent lifecycle
- Schedule tasks
- Handle retries
- Maintain execution state
- Emit workflow events

Core modules:

```
orchestrator/  
├── planner.py  
├── scheduler.py  
├── workflow.py  
├── approvals.py  
├── state_manager.py  
├── retry_manager.py  
└── event_dispatcher.py
```

11.6 LLM Router Service

Rather than binding to a single LLM provider, SDAP introduces an abstraction layer.

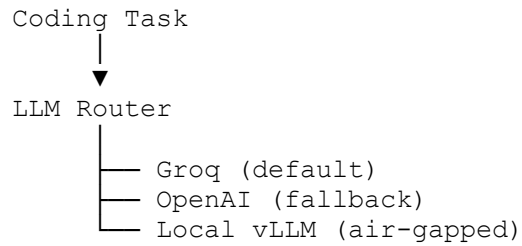
Supported providers (initial roadmap):

- Groq (Llama 3.3)
- OpenAI
- Anthropic
- Google Gemini
- Azure OpenAI
- Local vLLM

Routing policies:

- Model capability
- Token limits
- Cost
- Latency
- Availability
- Workspace preferences

Example:



11.7 Skill Registry Service

Responsibilities:

- Store skill metadata
- Version management
- Dependency resolution
- Search & discovery
- Access control

Suggested tables:

- skills
 - skill_versions
 - skill_dependencies
 - skill_permissions
-

11.8 Memory Service

The Memory Service centralizes context management.

Memory domains:

- Conversation memory
- Workspace memory
- Repository memory
- User preferences
- Long-term knowledge

Storage strategy:

Memory Type	Storage
Metadata	PostgreSQL
Semantic embeddings	Vector database
Session cache	Redis

11.9 Tool Execution Service

Responsibilities:

- Tool registration
- Tool routing
- Policy enforcement
- Sandboxed execution
- Result normalization

Internal modules:

```
tool_engine/  
├── registry.py  
├── router.py  
├── executor.py  
├── sandbox.py  
├── policy.py  
└── normalizer.py
```

11.10 MCP Client Service

The MCP Client Service abstracts communication with all external MCP servers.

Features:

- Server discovery
- Session pooling
- Authentication
- Health monitoring
- Retry logic
- Streaming responses

Supported transports:

- STDIO
- Streamable HTTP
- SSE

11.11 Repository Service

Responsibilities:

- Clone repositories
- Branch management
- Commit operations
- Diff generation
- Pull request preparation

Security:

- Read-only mode supported
 - Protected branch awareness
 - Signed commit support
-

11.12 Event Bus

Inter-service communication is event-driven.

Suggested technologies:

- Redis Streams (Phase 1)
- RabbitMQ (optional)
- Kafka (enterprise scale)

Example events:

```
TASK_CREATED  
PLAN_READY  
SKILL_LOADED  
TOOL_EXECUTED  
CODE_GENERATED  
TEST_COMPLETED  
PR_CREATED  
DEPLOYMENT_APPROVED
```

11.13 Background Processing

Long-running operations are delegated to background workers.

Candidate technologies:

Option	Use Case
Celery	Simple asynchronous jobs
Dramatiq	Lightweight task execution
Temporal	Durable workflow orchestration
Arq	Async Redis jobs

Recommendation: Use **Temporal** for enterprise deployments because it provides durable workflow execution, retries, and checkpointing that align well with long-running AI tasks.

11.14 Real-Time Communication

The frontend subscribes to execution progress through WebSockets.

Example event stream:

```
Task Started
Repository Indexed
Skill Loaded
Code Generated
Tests Running
Review Completed
Execution Finished
```

This provides developers with live visibility into agent execution.

11.15 Database Design (High-Level)

Primary database: PostgreSQL

Core entities:

- Users
- Workspaces
- Repositories
- Sessions
- Tasks
- Executions
- Agents

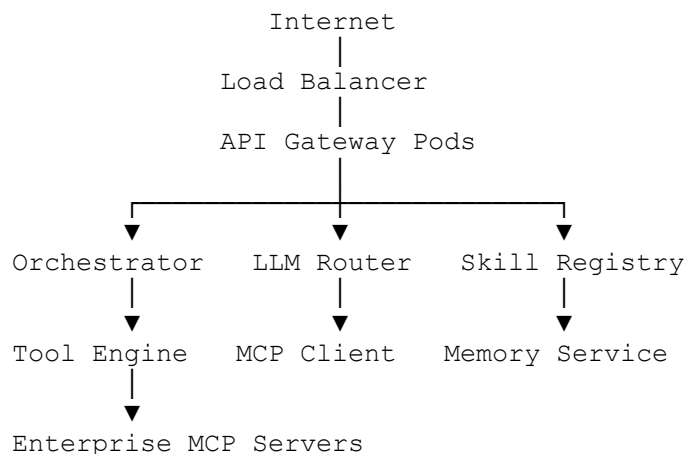
- Skills
- Tool Executions
- MCP Servers
- Audit Logs

Each service owns its schema, avoiding cross-service table coupling.

11.16 Infrastructure Components

Component	Purpose
PostgreSQL	Primary relational storage
Redis	Cache & session store
Vector Database	Semantic search
Object Storage	Artifacts & reports
Vault	Secret management
Prometheus	Metrics
Grafana	Dashboards
OpenTelemetry	Distributed tracing
Loki	Log aggregation

11.17 Deployment Topology



All services are deployed as independent Kubernetes Deployments with Horizontal Pod Autoscaling.

11.18 Security Architecture

Security controls include:

- OAuth2 / OpenID Connect
 - JWT validation
 - RBAC
 - Workspace isolation
 - Secret management via Vault
 - mTLS between services (optional)
 - Audit logging for all sensitive operations
-

11.19 Observability

Every service emits:

- Structured logs
- Metrics
- Traces
- Health checks
- Readiness probes
- Liveness probes

Dashboards include:

- Agent execution latency
 - LLM usage
 - MCP call latency
 - Tool execution success rate
 - Queue depth
 - Error rates
-

11.20 Architectural Decisions (ADR)

ADR	Decision	Rationale
ADR-039	Microservices architecture	Independent scaling and deployment
ADR-040	LLM Router abstraction	Multi-provider flexibility
ADR-041	Dedicated Memory Service	Centralized context management

ADR	Decision	Rationale
ADR-042	Event-driven communication	Loose coupling
ADR-043	Temporal-ready workflow model	Reliable long-running AI execution

11.21 Chapter Summary

The Backend Microservices & Infrastructure Design defines the runtime foundation of SDAP. By decomposing functionality into focused services, introducing an LLM Router, centralized memory, event-driven communication, and durable workflow support, the platform is positioned for enterprise-scale deployment and long-term evolution.

Chapter 12 – Code Generation Engine & Intelligent Software Development Workflow

Version: 1.0

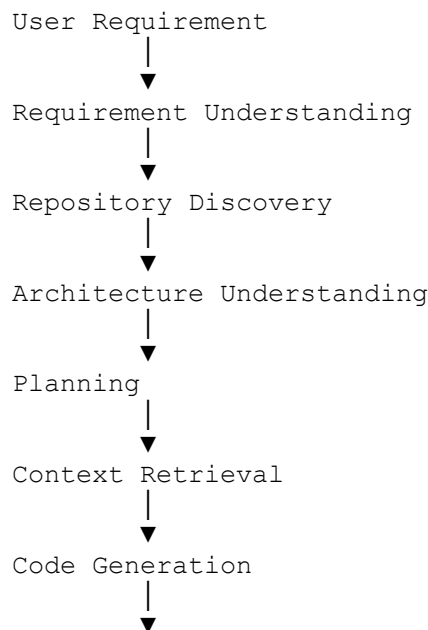
12.1 Introduction

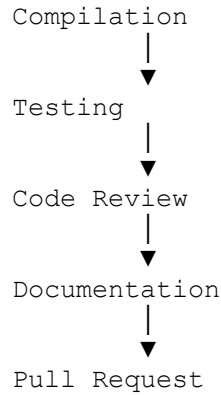
The Code Generation Engine (CGE) is the primary execution subsystem responsible for transforming natural language engineering requirements into production-ready software artifacts.

Unlike traditional AI coding assistants that directly generate code from prompts, the SDAP Code Generation Engine follows a structured engineering workflow involving repository understanding, semantic analysis, planning, iterative code generation, validation, testing, review, and documentation.

The objective is not merely to produce syntactically correct code, but to generate software that aligns with architectural standards, integrates seamlessly into existing repositories, and satisfies enterprise quality expectations.

12.2 Enterprise AI-SDLC Pipeline





Unlike traditional LLM systems, SDAP executes each stage independently.

12.3 Repository Intelligence Pipeline

Before writing any code, the platform constructs a semantic representation of the repository.

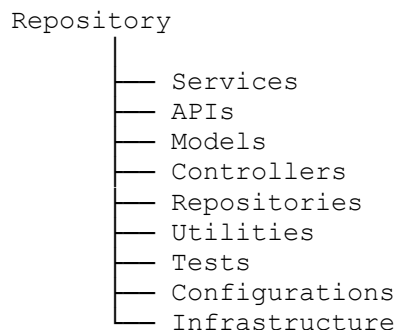
Stages:

1. File discovery
2. Language detection
3. Framework detection
4. Dependency graph construction
5. Symbol extraction
6. AST parsing
7. Import graph generation
8. API mapping
9. Database schema discovery
10. Test discovery
11. Configuration analysis
12. Security policy discovery

Outputs:

- Repository Graph
 - Dependency Graph
 - Service Graph
 - API Graph
 - Database Graph
 - Class Hierarchy
-

12.4 Repository Knowledge Graph



The graph is stored in the Memory Service and queried during planning.

12.5 Semantic Code Indexing

Every source file is indexed using multiple representations.

Representation	Purpose
Raw source	Exact retrieval
AST	Structural understanding
Tree-sitter parse tree	Incremental syntax analysis
LSP symbols	Semantic navigation
Embeddings	Semantic search
Dependency graph	Relationship analysis
Git history	Change context

This multi-layered index enables precise retrieval while minimizing context window usage.

12.6 Context Retrieval Strategy

Rather than loading the entire repository into the prompt, SDAP retrieves only the most relevant context.

Retrieval sources:

- Similar code

- Related APIs
- Affected modules
- Tests
- Design documents
- Previous implementation patterns
- Coding standards
- Architectural decisions

Ranking signals:

- Semantic similarity
- Dependency distance
- Git recency
- Symbol references
- Architectural importance

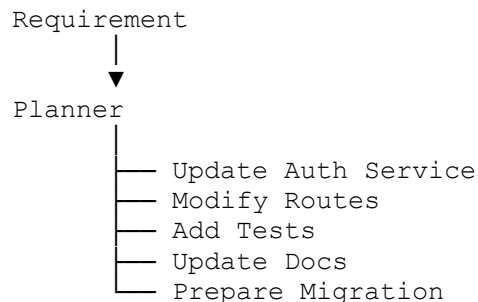
12.7 Planning Before Coding

The Coding Agent never writes code immediately.

Planning includes:

- Identify impacted modules
- Determine required interfaces
- Identify reusable components
- Assess backward compatibility
- Select implementation strategy
- Estimate affected tests

Example:



12.8 Intelligent Code Generation

The engine generates code incrementally rather than rewriting entire files.

Preferred operations:

- Insert function
- Modify method
- Update imports
- Add class
- Refactor block
- Generate tests
- Update documentation

This minimizes merge conflicts and preserves developer changes.

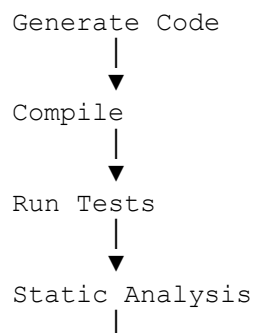
12.9 Language-Aware Generation

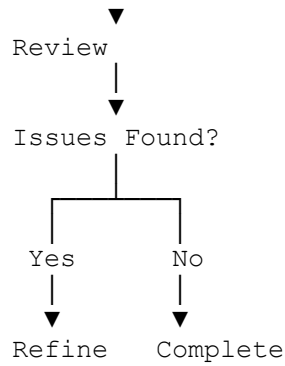
Generation strategies vary by language.

Language	Primary Analysis
Python	AST + Ruff + MyPy
Java	AST + Maven + Checkstyle
TypeScript	TypeScript Compiler API
Go	go/parser
Rust	rust-analyzer
C#	Roslyn

Each language integrates with its corresponding MCP server and language tooling.

12.10 Iterative Development Loop





The engine repeats this loop until validation criteria are met or configured retry limits are reached.

12.11 Self-Review Engine

After generation, the Reviewer Agent performs automated analysis.

Checks include:

- Coding standards
- Naming conventions
- Complexity
- Security vulnerabilities
- Dead code
- Duplicate logic
- Error handling
- Logging
- Performance considerations

Outputs:

- Review score
 - Findings
 - Suggested improvements
 - Risk assessment
-

12.12 Test Generation Strategy

The Testing Agent automatically generates:

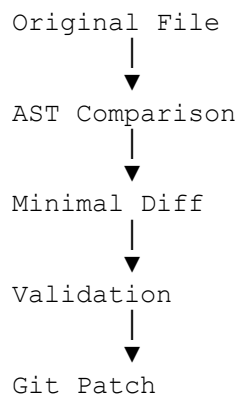
- Unit tests
- Integration tests
- API tests
- Regression tests
- Edge-case tests
- Negative tests

Coverage targets are configurable (e.g., 80%, 90%, 95%).

12.13 Intelligent Diff Generation

Instead of replacing complete files, SDAP generates precise patches.

Workflow:



Benefits:

- Smaller pull requests
- Easier reviews
- Reduced merge conflicts

12.14 Documentation Generation

Artifacts produced:

- Updated README sections
- API documentation
- Architecture notes
- Release notes

- Changelog entries
- Sequence diagrams (Mermaid)
- Inline code comments

Documentation remains synchronized with code changes.

12.15 Pull Request Preparation

Before opening a PR, SDAP compiles:

- Change summary
- Linked work items
- Test results
- Coverage report
- Review findings
- Breaking changes
- Migration notes

This creates a review-ready package for human approval.

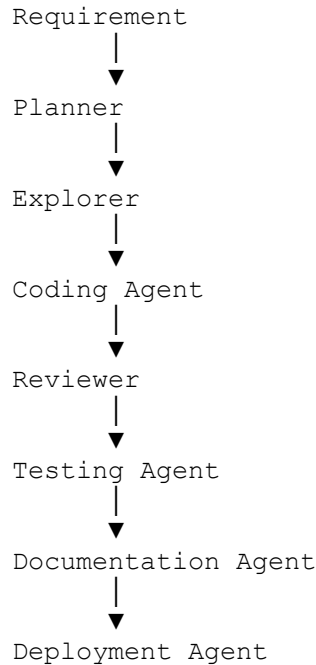
12.16 Quality Gates

Every change passes configurable quality gates.

Gate	Example Criteria
Build	Successful compilation
Tests	Minimum coverage threshold
Lint	No critical violations
Security	No high-severity findings
Architecture	Dependency rules satisfied
Documentation	Required updates completed

Execution proceeds only when mandatory gates pass.

12.17 Multi-Agent Collaboration



Each agent contributes specialized expertise while sharing execution context through the orchestration layer.

12.18 Intelligent Learning (Optional Future Capability)

Future versions may capture:

- Accepted code patterns
- Preferred architectural approaches
- Team-specific conventions
- Common review feedback

This information can be incorporated into future Skill versions without storing sensitive source code directly.

12.19 Architectural Decisions (ADR)

ADR	Decision	Rationale
ADR-044	Plan before code generation	Improves solution quality

ADR	Decision	Rationale
ADR-045	Multi-layer repository indexing	Efficient context retrieval
ADR-046	Minimal diff generation	Preserve developer changes
ADR-047	Iterative validation loop	Reduce defects before review
ADR-048	Language-specific analysis	Higher accuracy across ecosystems

12.20 Chapter Summary

The Code Generation Engine establishes an AI-native software development lifecycle that emphasizes repository understanding, planning, incremental modification, automated validation, and collaborative review. By combining semantic indexing, language-aware analysis, iterative refinement, and quality gates, SDAP moves beyond prompt-based code generation to deliver enterprise-ready software engineering workflows.

Chapter 13 – Enterprise Memory Architecture & Retrieval-Augmented Engineering (RAE)

Version: 1.0

13.1 Introduction

A Developer Agent cannot function effectively if every interaction begins with an empty context. Enterprise software development spans weeks or months, involving evolving requirements, architectural decisions, code reviews, testing, and deployment activities.

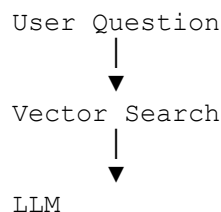
To support long-running engineering work, SDAP introduces **Retrieval-Augmented Engineering (RAE)**.

Unlike traditional Retrieval-Augmented Generation (RAG), which primarily retrieves documents to answer questions, RAE retrieves engineering artifacts, architectural knowledge, source code structure, repository history, execution state, and organizational standards to support intelligent software development.

The Memory Architecture provides persistent, context-aware knowledge across sessions, repositories, and teams while enforcing workspace isolation and governance.

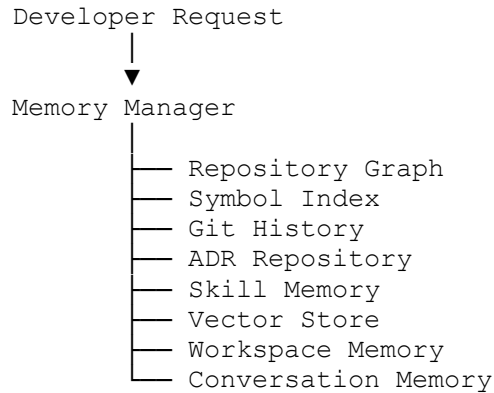
13.2 Why RAE Instead of Traditional RAG?

Traditional RAG:



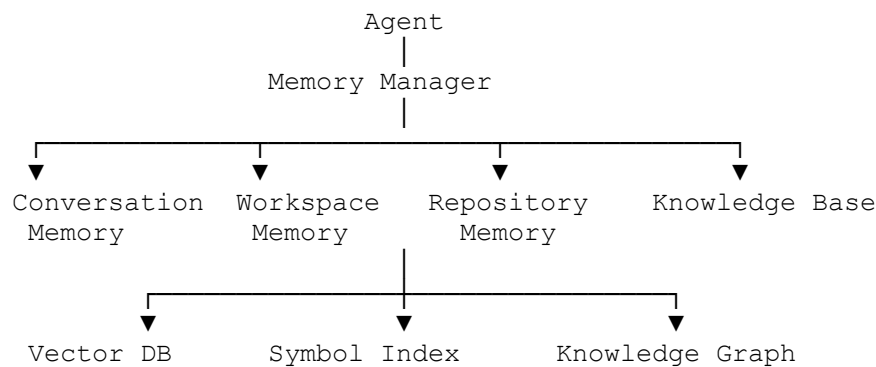
Useful for documentation, but insufficient for software engineering.

RAE:



The retrieved context is engineering-specific rather than document-centric.

13.3 Memory Architecture



13.4 Memory Domains

Conversation Memory

Stores:

- User requests
- Clarifications
- Agent responses
- Active execution state

Retention:

- Session-based with configurable persistence.

Workspace Memory

Stores:

- Workspace configuration
- Active repositories
- Environment variables (references only)
- Installed Skills
- Team preferences

Repository Memory

Stores:

- Dependency graph
- File hierarchy
- API inventory
- Class relationships
- Database mappings
- Build configuration
- Test inventory

Architectural Memory

Captures long-lived engineering knowledge:

- Architectural Decision Records (ADRs)
- Coding standards
- Design patterns
- Service boundaries
- Approved libraries

This ensures future work remains consistent with previous architectural decisions.

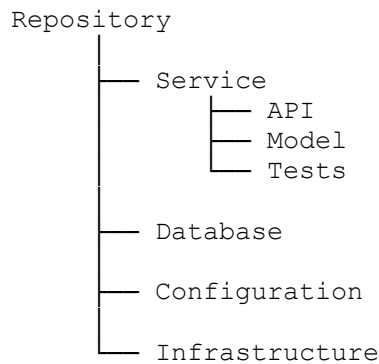
Skill Memory

Stores:

- Frequently used Skill combinations
- Skill execution statistics
- Validation outcomes
- Reusable engineering workflows

13.5 Knowledge Graph

Rather than relying solely on embeddings, SDAP maintains a graph of engineering entities.



Relationships enable structural reasoning that semantic embeddings alone cannot provide.

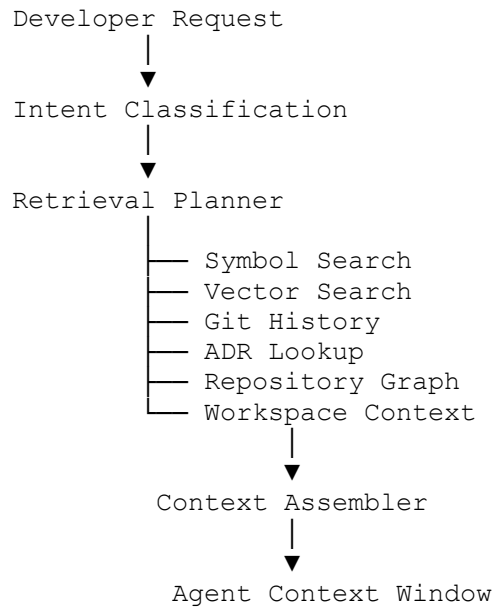
13.6 Hybrid Retrieval Strategy

RAE combines multiple retrieval techniques.

Retrieval Type	Purpose
Semantic (Embeddings)	Similar concepts
Lexical (BM25/Keyword)	Exact matches
Symbol Search	Functions, classes, variables
AST Navigation	Structural code retrieval
Dependency Graph	Impact analysis
Git History	Recent changes
Knowledge Graph	Architectural relationships

The Memory Manager merges results into a unified context package.

13.7 Context Assembly Pipeline



Only the most relevant information is injected into the agent's prompt.

13.8 Context Prioritization

The Context Assembler ranks information using weighted signals:

1. Current repository relevance
2. Direct symbol references
3. Architectural dependencies
4. Recent Git activity
5. Semantic similarity
6. Active workspace
7. User's current task
8. Historical execution outcomes

This minimizes token consumption while maximizing relevance.

13.9 Long-Term Memory

Long-term memory includes:

- Accepted architectural decisions
- Frequently reused implementation patterns
- Validated workflows
- Approved templates
- Shared engineering knowledge

Unlike conversation history, long-term memory persists across projects where organizational policy permits.

13.10 Memory Storage Strategy

Data Type	Storage
Metadata	PostgreSQL
Session cache	Redis
Embeddings	Vector Database
Knowledge graph	Graph database (optional)
Source artifacts	Object storage

This separation allows each storage technology to optimize for its specific workload.

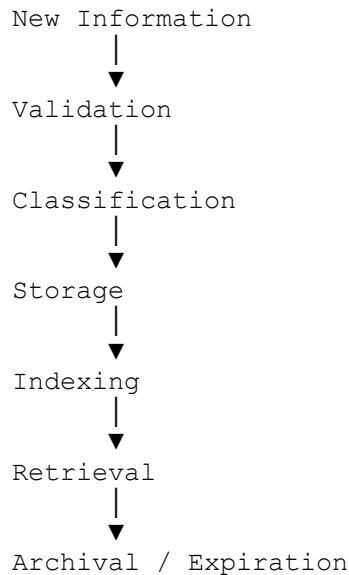
13.11 Memory Security

Security controls include:

- Workspace isolation
- Repository-level access
- Encryption at rest
- Encryption in transit
- RBAC enforcement
- Memory expiration policies
- Audit logging for retrieval events

Sensitive source code is never shared across unauthorized workspaces.

13.12 Memory Lifecycle



Only validated information is promoted into long-term memory.

13.13 Memory Update Policies

Memory is updated when:

- A task completes successfully.
- An ADR is approved.
- A pull request is merged.
- Documentation is accepted.
- Skills are upgraded.

Failed or rejected artifacts are retained only in execution history, not in reusable engineering memory.

13.14 Architectural Decisions (ADR)

ADR	Decision	Rationale
ADR-049	Retrieval-Augmented Engineering (RAE)	Engineering-aware retrieval beyond documents

ADR	Decision	Rationale
ADR-050	Hybrid retrieval model	Improve precision and recall
ADR-051	Knowledge graph support	Structural reasoning
ADR-052	Context prioritization	Optimize token efficiency
ADR-053	Memory domain isolation	Security and governance

13.15 Chapter Summary

The Enterprise Memory Architecture transforms SDAP into a persistent engineering platform capable of maintaining architectural continuity across long-running software projects. By combining hybrid retrieval, knowledge graphs, repository intelligence, and domain-specific memory, Retrieval-Augmented Engineering enables agents to make contextually accurate and architecturally consistent decisions that go far beyond traditional document retrieval systems.